

毕业论文（设计）

题目：基于 AI Agent 的高性能分布式通讯系统

学 生 姓 名： 以琳

学 号： _____

班 级： _____

专 业： _____

指 导 教 师： _____

2026 年 5 月

基于 AI Agent 的高性能分布式通讯系统

学 生 姓 名: 以琳

学 号: _____

班 级: _____

所 在 学 院: _____

指 导 教 师: _____

完 成 日 期: _____

诚 信 声 明

本人声明：

- 1、本人所呈交的毕业设计（论文）是在老师指导下进行的研究（设计）工作及取得的研究（设计）成果；
- 2、据查证，除了文中特别加以标注和致谢的地方外，毕业设计（论文）中不包含他人已经公开发表过的研究成果，也不包含为获得其他教育机构的学位而使用过材料；
- 3、我承诺，本人提交的毕业设计（论文）中的所有内容均真实、可信。

作者签名：_____

日 期：_____ 年 ____ 月 ____ 日

摘要

随着大语言模型与工具调用技术的发展, AI Agent 已从单轮问答工具演进为能够理解目标、分解任务、调用外部工具并持续反馈的自治执行单元。多 Agent 协作时, 系统需同时处理任务编排、跨进程通讯、状态一致性、执行隔离、实时观测与故障恢复等工程问题。现有多智能体框架以 Python 平台与角色对话流程为主, 在本地化、高性能、可审计的分布式通讯底座方面仍有较大空间; 尤其是 Agent2Agent (A2A) 协议虽已有官方 Rust SDK (a2aproject 维护的 a2a-rs) 与若干第三方 crate, 但缺少与本地优先平台的事件总线、编排层与人工介入语义深度集成的实现。

针对上述问题, 本文设计并实现一套名为 Fuxi 的 Rust 平台。系统采用「玄女—门客」分层协作模式将面向用户的顶层 Agent 与执行任务的工作 Agent 解耦; 自主从零实现一份面向 Fuxi 场景的 A2A 风格核心 RPC 闭环 (覆盖 agent discovery、task send、task stream、task query 与 task cancel 五条主路径, 沿用早期 A2A JSON-RPC binding), 并把 A2A 规范已有的 TASK_STATE_INPUT_REQUIRED 人工介入语义 (在本工作沿用的早期 binding 中编码为 "input-required") 映射到 Fuxi 内部 `Task::PendingApproval` 与 `ShelfStatus::AwaitingInput`, 使人工介入成为平台级可观测事件; 以 Tokio broadcast 与 SQLite WAL 构建实时事件总线, 使实时推送与历史回放在同一抽象上共存; 以 Firehose、WebSocket、SSE 与 IM API 提供多端观测; 以 Git worktree 与三层 sandbox 保证任务执行隔离。论文进一步给出端到端延迟分解、广播事件延迟下界与 Worker 选择函数的形式化定义, 使系统性能具备可量化、可比较的分析基础。

实验环节在 Apple Silicon 平台上对吞吐量、延迟、参数敏感性与事件总线压力进行了多维测量。结果表明, 在剥离 LLM 推理后的合成任务基准下 (每任务以 10 ms sleep 模拟), Fuxi 在 8 worker 配置下达到 665.00 tasks/s 的调度路径吞吐, 扩展至 16 worker 时增长至 1288.24 tasks/s (1→16 worker 区间扩展效率介于 80.5%–83.1%, dispatcher 在该区间未饱和); 任务派发链路 p50 延迟为 32.93 ms (含 HTTP / SQLite / 反序列化等组件), 进程内事件总线广播 p99 延迟为 0.26 ms; 事件总线在 64 subscriber 并发订阅、10 000 events/s 持续负载下保持零丢帧。需要说明的是, 上述合成基准衡量的是编排与通讯层在剥离 LLM 推理后的纯通讯吞吐上限, 真实 LLM Agent 任务的端到端吞吐受单次推理 (典型 1–30 s) 主导。这些数据从侧面验证了「事件驱动与追加式日志结合的通讯架构能够在本地优先的 AI Agent 协作场景中同时兼顾吞吐、实时性与可追溯性」的设计假设, 为后续多 Agent 系统的工程化提供了一套可借鉴的基础设施。

关键词: AI Agent; 分布式通讯; 多智能体系统; 事件总线; A2A 协议

ABSTRACT

With the rapid progress of large language models and tool-use techniques, AI agents are evolving from single-turn assistants into autonomous execution units that understand goals, decompose tasks, invoke external tools and continuously report progress. When multiple agents collaborate, a platform must simultaneously address task orchestration, inter-process communication, state consistency, execution isolation, real-time observability and fault recovery. Existing multi-agent frameworks are primarily built on Python platforms with role-driven dialogue flows, leaving room for improvement in local-first, high-performance and auditable distributed communication infrastructure. In particular, although Agent2Agent (A2A) already has an official Rust SDK (`a2a-rs`, maintained by `a2aproject`) alongside several third-party crates, none of them deeply integrates with a local-first orchestration layer, an event bus and human-in-the-loop semantics, which constrains the deployment of audit-friendly, observable Rust-based agent platforms.

This thesis designs and implements Fuxi, a Rust-based platform that addresses these gaps. Fuxi adopts a hierarchical Xuanny-Menke collaboration pattern that separates the user-facing orchestrator agent from worker agents. Fuxi provides a from-scratch Rust implementation of an A2A-style core RPC closure (covering agent discovery, task send, task stream, task query and task cancel via the early A2A JSON-RPC binding) co-evolving with the platform’s event bus and orchestrator, and lifts the A2A specification’s existing `TASK_STATE_INPUT_REQUIRED` task state (encoded as `"input-required"` in the early binding adopted by this work) into a platform-level human-in-the-loop signal mapped onto the orchestrator’s `ShelfStatus::AwaitingInput`. A real-time event bus is built on top of Tokio broadcast and SQLite WAL so that live subscription and historical replay coexist on the same abstraction. Observability is exposed through Firehose, WebSocket, SSE and IM APIs, and execution is isolated through Git worktrees and a three-layer sandbox. The thesis further formalizes end-to-end latency decomposition, broadcast event latency lower bounds and a Worker selection function, providing a quantifiable basis for performance analysis.

Experimental evaluation on an Apple Silicon platform measures throughput, latency, parameter sensitivity and event bus stress along multiple dimensions. Under a synthetic benchmark in which each task is modeled as a 10 ms sleep (isolating the orchestration layer from LLM inference), Fuxi sustains 665.00 tasks per second with 8 workers, and scales to 1288.24 tasks per second with 16 workers, with the 1-to-16-worker range exhibiting 80.5%–83.1% scaling efficiency and the dispatcher remaining unsaturated within this range. The p50 task dispatch latency is 32.93 ms (covering HTTP, SQLite and deserialization), and the p99 in-process event-bus fan-out latency is 0.26 ms. With 64 concurrent subscribers, the event bus sustains 10 000 events per second with zero drops. These synthetic numbers measure the throughput ceiling of the orchestration and communication layers after factoring out LLM inference; end-to-end throughput under real LLM Agent tasks is dominated by per-inference latency (typically 1–30 s). Collectively, the results validate the design hypothesis that event-driven semantics combined with

append-only logging delivers a communication substrate that simultaneously achieves throughput, real-time responsiveness and traceability for local-first AI agent collaboration, providing a reusable foundation for future multi-agent system engineering.

Key words: AI Agent; Distributed Communication; Multi-Agent System; Event Bus; A2A Protocol

Contents

1	绪论	1
1.1	研究背景及意义	1
1.2	国内外研究现状	2
1.3	研究目标与贡献	3
1.4	论文组织结构	4
1.5	本章小结	4
2	系统总体设计	4
2.1	设计目标与约束	4
2.2	总体架构	6
2.3	关键数据流	8
2.4	跨节点扩展	9
2.5	SLA 与性能预算	11
2.6	本章小结	11
3	系统设计与实现	12
3.1	模块布局与 13 crate 边界	13
3.2	事件总线模块	14
3.3	A2A 协议模块（论文核心贡献）	16
3.4	编排与任务派发模块	19
3.5	执行适配器与三层沙箱	21
3.6	跨节点通讯实现	22
3.7	观测、IM 与横向支撑模块	23
3.8	本章小结	24
4	系统测试与结果分析	24
4.1	测试目标与方法学	24
4.2	测试用例覆盖	25
4.3	吞吐量、扩展性与端到端延迟分解	26
4.4	延迟分布与尾部分析	28
4.5	poll_ms 参数消融	30
4.6	事件总线压力承受能力	31
4.7	功能验收与本章小结	32
5	总结与展望	33
5.1	工作总结	33
5.2	局限与不足	34
5.3	未来工作	35
5.4	结语	36

5.5 本章小结	36
致谢	39

1 绪论

伏羲是一套以 Rust 实现的本地优先 AI Agent 编排平台。它把用户的自然语言意图、顶层 Agent 玄女的对话决策、多个并发执行的门客（基于 Claude Code 与 Codex 等 headless agent CLI 启动的工作进程）以及它们之间的协作消息，统一视作可观测、可回放、可审计的事件序列；通过单一 SQLite WAL 真相源、Tokio broadcast 实时分发与 Google A2A 协议互通三套基础设施，把多 Agent 系统从「演示脚本」推到「日用平台」。本章梳理伏羲所处的技术语境与同类系统现状，明确论文要解决的问题与五条贡献，并给出全文章节安排。

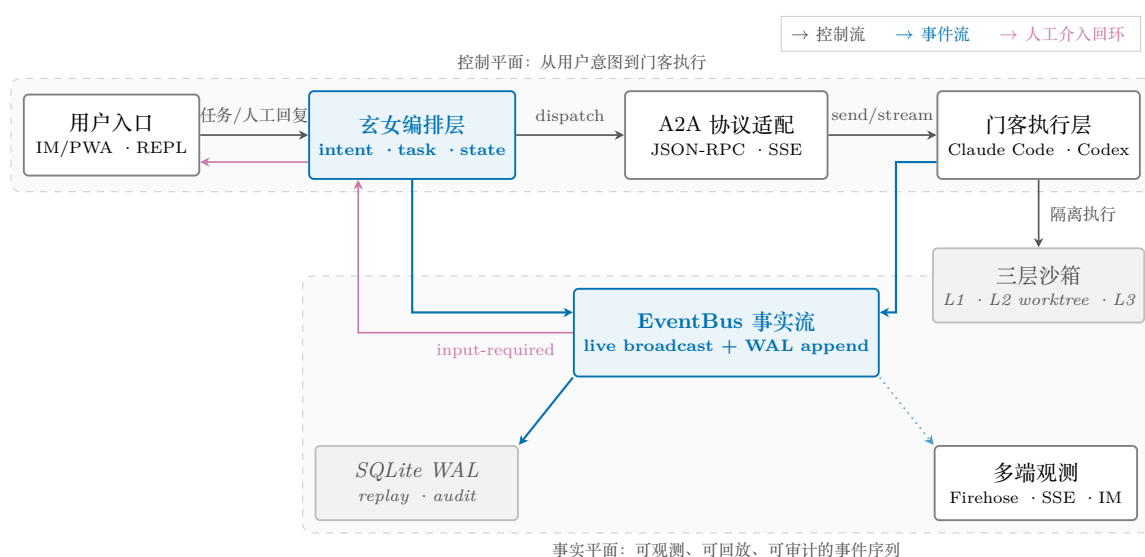


图 1.1 伏羲平台总体示意

1.1 研究背景及意义

AI Agent 的现代理论基础源自 Wooldridge 提出的「自治、社会能力、反应性、主动性」四属性[1] 与 Russell 等的「感知—决策—行动」闭环[2]。大模型成熟之后，这一抽象层次被迅速工程化：Wang 等的综述把现代 Agent 抽象为「规划、记忆、工具使用、行动反馈」四组件[3]；ReAct[4]、Toolformer[5]、MRKL[6] 让 Agent 具备显式工具调用能力；Voyager[7] 通过 skill library 沉淀跨任务经验；SWE-agent[8] 与 OpenHands[9] 把执行环境与 Agent-Computer Interface 提升为一等公民。这些工作共同表明，工具接口、执行隔离与持续记忆必须作为系统组件设计，而不是对话循环的附属物。

工程使用中，单 Agent 难以同时承担需求澄清、计划拆解、代码实现与结果交付。CAMEL[10]、AutoGen[11]、MetaGPT[12]、AutoGPT[13] 等多 Agent 框架证明角色分工能提升任务可解释性，但也暴露三类共性工程问题：通讯协议不统一、任务生命周期与持久化缺位、运行时状态难追踪。对一个本地化运行、要被日用的多 Agent 平台而言，高性能分布式通讯不是可选锦上添花而是平台前提——Agent 执行过程会产生密集的工具调用与中间消息，需要低延迟通道及时反馈给用户；多 Agent 并发调度不能依赖轮询；Agent 输出的概率性需要事件日志、会话恢复与执行隔离来降低不可追溯风险。这与 Lamport 在 1978 年提出的事件因果序思想[14] 在工程上同构：一个看不到事件历史

的多 Agent 系统形同一个没有时钟的分布式系统。

2024 年以来 headless agent CLI 的兴起把上述需求具象化：`claude -p`、`codex exec` 等工具以「整段任务一次性输入、过程事件流式输出」的形态投入生产，可被 shell 脚本、CI 流水线或上层 Agent 调用。多 Agent 协作不再是同一进程内的对象互调，而是多个独立子进程之间的结构化消息调度，这与分布式系统「以消息为契约、以进程为隔离单元」的传统范式重合。本工作面向「LLM agent CLI 已成既定事实、缺一个把通讯底座当一等公民设计、能在普通笔记本上稳定承载日常工作流并可跨节点扩展的平台」这一需求展开。

为方便后文阅读，先约定术语：顶层调度 Agent 称「玄女」，对应 `fuxi-orchestrator`；执行任务的工作 Agent 称「门客」，适配器为 `fuxi-agent-cc/fuxi-agent-codex`；事件分发抽象称「事件总线」，对应 `EventBus`。Google 主导的 Agent2Agent 协议简称 A2A，Anthropic 的 Model Context Protocol 简称 MCP——前者描述 Agent 间协作，后者描述模型与工具的接口，二者面向不同语义层次。

1.2 国内外研究现状

围绕 AI Agent 编排已形成几条平行研究路线。**Agent** 行为模式以 ReAct[4]、Toolformer[5]、MRKL[6] 为代表，关注单 Agent 推理—行动—观察闭环的能力上限，但通讯层退化为函数调用，不涉跨进程协作。多 **Agent** 协作框架以 AutoGen[11]、MetaGPT[12]、CAMEL[10]、AutoGPT[13] 为代表，证明角色分工有效，但绝大多数以 Python 框架或云端 SaaS 为形态：消息走进程内对象图、状态绑死生命周期、不能跨节点扩展。后续工作如 ChatDev[15]、CrewAI[16] 与 AutoGen v0.4[17] 仍延续 Python+角色对话主流路线，与本文 Rust + 跨进程门客 + 跨节点 A2A 路线在底层选择上分道。

Agent 协议与编排标准是另一条线。MCP[18] 标准化模型与工具接口，A2A[19] 用 `AgentCard`、`Task`、`Message` 等概念描述 Agent 间互操作语义。A2A 已有 Python、Go、Java、JavaScript、.NET 与 Rust (`a2a-rs`) 六语言官方 SDK，Rust 生态内还有 `a2a-client`、`a2a-types` 等第三方 crate。本工作不依托「Rust 生态空白」立论，而是面向「与本地优先编排层和事件总线深度集成、并把 `input-required` 语义映射为平台级人工介入状态」这一具体场景实现一份与平台同源演进的 A2A 风格协议适配层。分布式系统经典工作——Lamport 逻辑时钟[14]、Kafka 追加式日志[20]、Raft[21]、ZooKeeper[22]——虽不针对 Agent，但其追加式日志、push 式分发与故障恢复思想在伏羲事件总线中被反复借鉴。Parrot[23] 在 LLM 推理服务层关心 GPU 集群效率，与本工作在多进程编排层的关注点正交但互补。

工程基础设施已经齐备：Tokio 提供异步运行时与 `broadcast/mpsc channel`[24]，Axum 基于 tower/hyper 提供 Web 框架[25]，git worktree 提供轻量任务隔离[26]，WebSocket[27] 与 SSE[28] 支撑实时通讯。Rust 生态围绕 LLM 的 `llm-chain`、`rig`、`kalosm` 等库多停留在「单 Agent 调模型 + 工具调用」层级，没有覆盖多 Agent 协作、跨节点调度或事件溯源这类系统级议题。同类 Python 系统普遍存在三类共性盲区：GIL 与协程切换让多核难以充分利用、上下文绑死 Python 对象图崩溃即丢、没有跨节点能力。伏羲选择 Rust+多进程门客+跨节点 A2A 是有意识地避开这些盲区。

1.3 研究目标与贡献

本文研究目标是：在 Rust 工作区中实现一个本地优先、事件驱动、可观测、可跨节点扩展的多 AI Agent 协作平台，把通讯底座作为一等公民设计，并通过吞吐、延迟与压力测试给出可量化性能边界。围绕这一目标做出五项可独立审视的工程贡献。

贡献一：与平台同源演进的 **A2A** 风格协议适配层。fuxi-a2a crate 共 1321 行（核心 1063 + 测试 258），覆盖 agent/getCard、tasks/send、tasks/sendSubscribe、tasks/get、tasks/cancel 五条核心路径，沿用早期 A2A JSON-RPC binding。差异化贡献集中在两点：把规范已有的 input-required 中间态从协议字段提升为平台级人工介入信号——映射到 Task::PendingApproval 与 ShelfStatus::AwaitingInput；采用单端点 HTTP+SSE 设计避免客户端管理两条独立连接。本实现采用早期 binding，与当前官方 PascalCase 方法集和 Part one-of 语义不完全对齐，亦未与 a2a-rs 做完整 wire-level 互通测试，故不主张与 v1.0 规范完全对齐。

贡献二：非阻塞事件总线与 **lag** 哨兵机制。fuxi-events 把实时分发、持久化与历史回放统一在同一抽象内。publish() 路径采用「try_send + 后台转交 + 哨兵告警」三步组合：广播路径用 Tokio broadcast 零拷贝扇出，持久化路径 try_send 投递 mpsc 队列、拥塞时后台 spawn 任务承担等待；当 writer 待处理数超过 lag_threshold = 512 时发出 Custom { "event_store_lagged" } 哨兵事件，让运维侧把背压从「不可见的死锁」看成「可见的告警」。该设计兼顾「调用方永不阻塞」与「事件不丢」两个看似矛盾的约束，是对 Kafka push 模式[20] 在本地优先场景下的工程化适配。

贡献三：玄女一门客分层与不可绕过的编排层。系统在 Agent 角色上确立顶层调度（玄女）与执行 Agent（门客）的显式分工。玄女是用户对话面唯一入口，所有门客之间通讯必须经玄女知情，任何调度决策都发出对应事件让观测端透明。这条「编排层不可绕过」硬约束在 Fuxi::shutdown_agent 等敏感操作中反复实施：所有 shutdown 路径必须比对玄女 AgentId，命中则 warn 并 noop，例外仅限显式命名 shutdown_xuannv_for_handoff 的会话交接路径。Worker 选择函数沿「角色匹配、节点钉死、最低饱和度」三层过滤逐级缩小候选集。

贡献四：三层沙箱 × **WebSocket** 反连门客执行模式。针对多 Agent 并发的两个核心难题——文件系统隔离与进程间双向通讯——本文提出 L1/L2/L3 三层沙箱模型（只读视图 / 任务级 worktree / 角色级持久沙箱），依赖 git worktree[26] 提供互不干扰的工作树；同时提出 WS 反连模式——编排层在本机启 WebSocket 服务器，子进程通过 --sdk-url 反连回来，使编排层在任意时刻都能主动推送 send_message 或 cancel，不必等门客 poll stdio。这两项设计共同把「多 Agent CLI 适配」收敛到统一框架，已支持 Claude Code 与 Codex 两套差异极大的 headless agent CLI。

贡献五：跨节点扩展点的钩子化抽象与 **sandbox** 验证。跨节点协作所需的扩展点抽象为四个依赖反转 trait: RecallSink、DistEnqueuer、NodeLoadProvider、ExtractorSpawner——统一定义在低层 crate，由顶层 fuxi-cli 在 daemon 启动时注入，规避「memory 反向依赖 orchestrator」之类循环依赖。Project 数据结构新增 host_nodes: Vec<String> 标记可执行节点，配合 auto_pin_from_project 入口的双保险守卫保证用户显式 @<node> 不被自动逻辑覆盖；新增字段以

`#[serde(default)]` 搭配反回归测试保持跨版本兼容。当前实现已在两节点 sandbox 与少量真实跨机部署中跑通端到端调度，更高节点数与不可靠链路下的故障转移留待后续工作（详见 §6.2）。

上述五项贡献围绕同一命题——把 Agent 间通讯的契约、事实流、调度边界与执行隔离同时做对——从协议层到平台层逐级展开。本文最终交付一份 13 crate、约 8.25 万行 Rust 代码的可执行工作区，配套 1363 个单元测试、27 个集成测试与持续运行于作者 ERP 项目的真实业务验证。

1.4 论文组织结构

全文按「问题—方案—实现—验证—回顾」展开，共五章。第一章（本章）阐述背景、研究现状、目标与五项贡献。第二章系统总体设计从功能与非功能需求出发，给出整体分层架构与通讯模型形式化定义，明确事件驱动、单一真相源、不可绕过编排三条公理，并定义吞吐量、端到端延迟等性能指标。第三章系统设计与实现沿 13 个 crate 模块依赖自下而上展开，覆盖事件总线、A2A 协议、玄女—门客调度、执行适配器与三层沙箱、跨节点通讯，把每个模块的关键数据结构、接口签名与代码片段一并呈现。第四章系统测试与结果分析沿正确性、实时性、吞吐扩展性、参数敏感性、通讯层压力承受五个维度组织实验，吞吐采用 5-run median、延迟采用 500 样本统计 p50/p99。第五章总结与展望回顾全文工作并讨论分布式一致性、记忆语义召回、Agent 适配器扩展、安全模型与用户界面五个方向的不足与后续路径。

1.5 本章小结

本章梳理 Agent 理论与 LLM 工程化两条线索，确立「把通讯底座作为一等公民」的核心立场；横向考察行为模式、协作框架、协议标准与分布式经典工作四条研究路线，指出现有同类系统在 Python 中心、单进程状态、跨节点缺失三方面的共性盲区；给出五项工程贡献——A2A 风格协议适配层、非阻塞事件总线与 lag 哨兵、玄女—门客分层、三层沙箱与 WS 反连、跨节点扩展点钩子化抽象。下一章把这些贡献映射到具体模块边界与形式化指标定义。

2 系统总体设计

第一章把多 Agent 协作场景的工程缺口归结为「真实时观测、跨 CLI 异构、单机本地优先、跨节点可扩展、历史可回放」五条诉求。本章把这五条诉求落到伏羲（fuxi）的总体架构上：自顶向下给出设计目标与约束、整体结构、关键数据流、跨节点扩展与性能预算。本章不展开 Rust 实现，只把架构层面的取舍——为何选择事件驱动而非 RPC 主导、为何把 A2A 与事件总线视为正交、为何编排层在玄女与门客之间硬性插一层而非走对等图——逐项说明。

2.1 设计目标与约束

伏羲的总体设计需化解三组矛盾。第一组「真实时」与「不丢消息」——观测端要求事件抵达延迟尽可能低，审计与回放要求事件不能在拥塞时被悄悄丢弃；第二组「多门客并发」与「文件系统隔离」——多个 CLI 门客同时改一个项目目录会撞 git 锁，而完全

独立的目录又会让 `cargo build` 与 `node_modules` 这类大型构建产物每次重生；第三组「跨节点扩展」与「单机本地优先」——分布式调度要求节点间共享调度状态，本地优先又要求单节点离线时仍能完整运行。

第一组通过事件总线的双路径架构解决：事件在每次 `publish()` 同时进入广播路径（Tokio broadcast[24] 零拷贝扇出）与持久化路径（异步 mpsc 队列由后台 writer 串行落库到 SQLite WAL[29]）；writer 拥塞时不阻塞调用方，而是 spawn 后台任务等待，并在堆积超阈值时发出哨兵事件让运维侧感知。该设计借鉴 Kafka 的日志驱动事件流经验[20]，但不引入 Kafka 这种重型外部组件。第二组通过 git worktree[26] 多工作树机制与 L1/L2/L3 三层沙箱解决：多个门客并发执行时各自在独立 worktree 上写改动，最终合并回主线；只读场景跳过 worktree 创建直接挂载；持久角色级沙箱跨任务复用构建缓存。第三组通过「单机优先 + 钩子化扩展」解决：系统默认在单节点完整运行，所有持久状态都在本地 SQLite；当用户启动多节点部署时，`fuxi-cli` 注入 `DistEnqueuer` 与 `NodeLoadProvider` 两个 trait 对象到编排层，编排层自身不感知分布式细节。

把功能与非功能需求与设计决策做成可校对清单，列于表 2.1。这张映射表强制设计决策对需求负责，避免「为做而做」的过度工程。

表 2.1 功能/非功能需求与设计决策映射

需求类别	具体诉求	对应设计决策
多 Agent 调度	玄女把任务派给合适门客；同角色多门客并发；异构 CLI 统一调度	subsection 2.2 玄女—门客分层；subsection 2.3 dispatch pump
真实时观测	用户在 TUI/IM/PWA 看到任务进展无轮询；事件 $P99 \leq 50\text{ ms}$	EventBus + Firehose；subsection 2.5 公式 (2-4)
单一真相源	所有事件 append-only 落 SQLite；崩溃后能完整重放	双路径架构；Replay-Cursor 接口
工作区隔离	多门客并发不撞 git 锁；构建缓存可复用；只读场景轻量化	L1/L2/L3 三层沙箱
人工介入	门客遇到歧义可悬挂等待；用户回复后能续跑	A2A input-required 平台映射
跨节点扩展	跨机部署门客；按节点负载自动选择；用户显式 @<node> 钉死	subsection 2.4 host_nodes + auto-pin + 钩子注入
历史回放	任意时间点开始重放；回放与 live tail 无缝衔接	EventStore + 先订阅后查询
长期记忆	Agent 间共享事实与角色技能；扩展独立模块不绑主链	fuxi-memory/fuxi-skills 横向接入
轻量部署	单二进制；无外部数据库；单机离线可用	SQLite + Tokio + Axum 自包含栈
可观测背压	拥塞不悄悄丢消息；运维侧能看到滞后趋势	lag 哨兵事件；公式 (2-4)

除上表外还有两条隐性约束：所有 Agent 间协作必须经编排层显式记录，不允许 Agent 直接 RPC 绕过玄女——这是公理「玄女永远有知情权，无否决权」的工程化表达；观测层只读不写，任何业务状态变化不允许从观测端反向修改主链——这条让观测端可被替换或并存（同一时刻浏览器、TUI、PWA 三端共看一份事件流）而不破坏正确性。

2.2 总体架构

伏羲采用「核心—通讯—编排—执行—观测」五层架构，每层只依赖更下一层暴露的稳定接口，依赖方向严格自下而上，禁止循环引用。整体如图 2.1 所示。

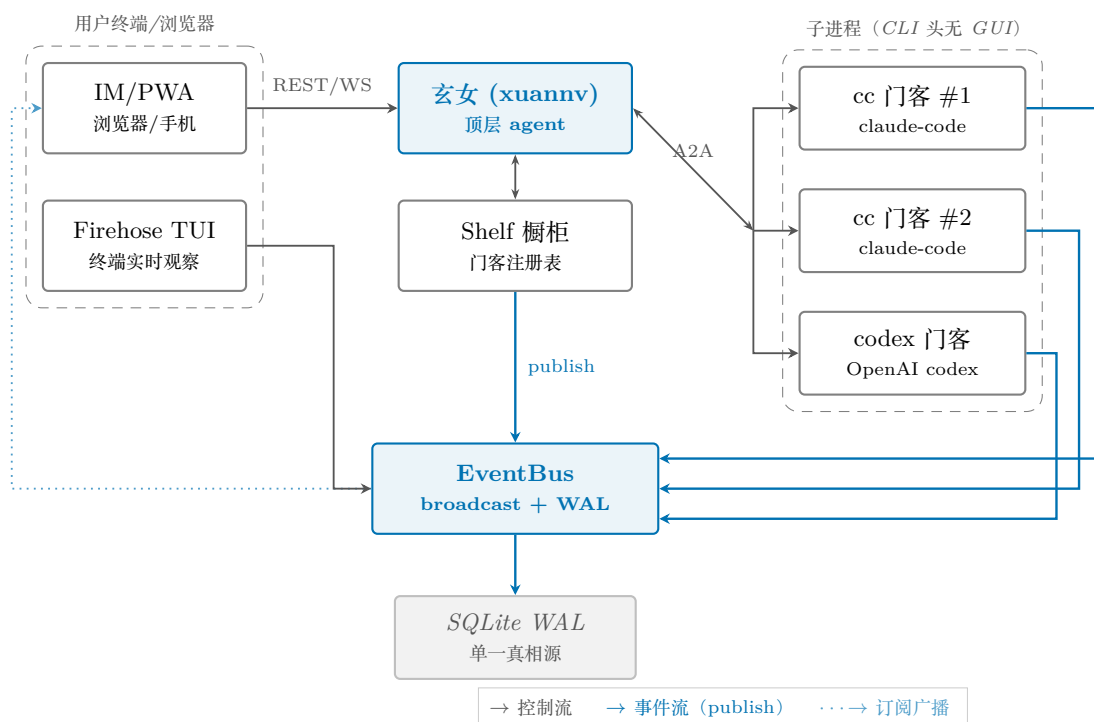


图 2.1 伏羲系统总体架构（五层 + 横向支撑模块）

最底层核心层（fuxi-core）集中定义 Agent、Workspace、Event、Task 四个 trait 与对应值对象，不引入任何外部 IO。通讯层由 fuxi-events（事件总线）与 fuxi-a2a（A2A 协议）共同构成，两者职责互补但语义正交——事件总线描述「系统内部已经发生了什么」，是事实流；A2A 协议描述「Agent 之间应当传递什么」[19]，是契约语义。这一边界划分是伏羲与 AutoGen[11]、CAMEL[10] 这类把协议与状态合并的系统最显著的差异：在伏羲中 A2A 信封只承担最小契约，所有运行时状态都走事件总线。

编排层（fuxi-orchestrator）是中枢，维护门客注册表 Shelf、把任务派给合适的门客、提供事件 republish 通路让门客私有事件流进入公共总线。该层借鉴 MetaGPT[12] 的角色分工但更强调「编排不可绕过」——所有门客之间通讯必须经编排层透明记录。执行层由两个适配器（fuxi-agent-cc、fuxi-agent-codex）与隔离层（fuxi-workspace）构成：cc 适配器走 WebSocket 反连[27] 长连接复用进程，codex 适配器走 spawn-per-dispatch；隔离层基于 git worktree 提供 L1/L2/L3 三层沙箱。观测与服务层由 fuxi-firehose（事件流四端输出）与 fuxi-im（IM 后端）构成：Firehose 把同一份 EventBus 通过 WebSocket、SSE[28]、REST 与终端 TUI 四种形态对外暴露；IM 后端面向 PWA 与桌面客户端提供 REST 任务管理与双向消息流。两者均对编排层只读不写。

横向支撑模块（fuxi-memory、fuxi-skills、fuxi-scheduler）通过事件订阅与显式 API 与编排层交互，自身不参与主链调度。fuxi-memory 提供长期记忆，思路源自 Voyager 持久技能库[7]；fuxi-skills 是角色玉牒加载器；fuxi-scheduler 提供 cron/once/webhook/文件变化四类触发器，统一通过发出 TriggerFired 事件接入主链。最顶层 fuxi-cli 把所有 crate 装配成可执行二进制 fuxi，内部分成 REPL、

daemon、dist、im 等十余个子模块。

13 个 crate 总代码规模约 8.25 万行，每个的形式化边界、职责、依赖范围与具体实现细节在第三章 §3.1 模块布局中以表格形式列出，本章只点出依赖关系约束：若令 C 表示所有 crate 集合，关系 $c_i \rightarrow c_j$ 表示 c_i 依赖 c_j ，则系统约束 $\forall c_i, c_j \in C : (c_i \rightarrow c_j) \wedge (c_j \rightarrow^* c_i) \Rightarrow \perp$ （依赖图无环）。这条由 Cargo 在编译期强制检查，比注释规约或代码审查更可靠。fuxi-memory 的「召回结果回传」需要 orchestrator 持有的事件总线，但若让 memory 反向依赖 orchestrator 会形成循环——本设计采用「trait 在低层、impl 在高层」的反向依赖：在 orchestrator 中定义 RecallSink trait，由顶层 fuxi-cli 提供实现并注入。这是规避循环依赖的标准 Rust 工程惯用法，伏羲在四个跨 crate 钩子上一致采用。

2.3 关键数据流

为把架构层面的静态分层映射到运行时动态行为，本节沿一次完整用户请求展示核心数据流。图 2.2 给出从用户输入到事件多端扇出的核心数据流。

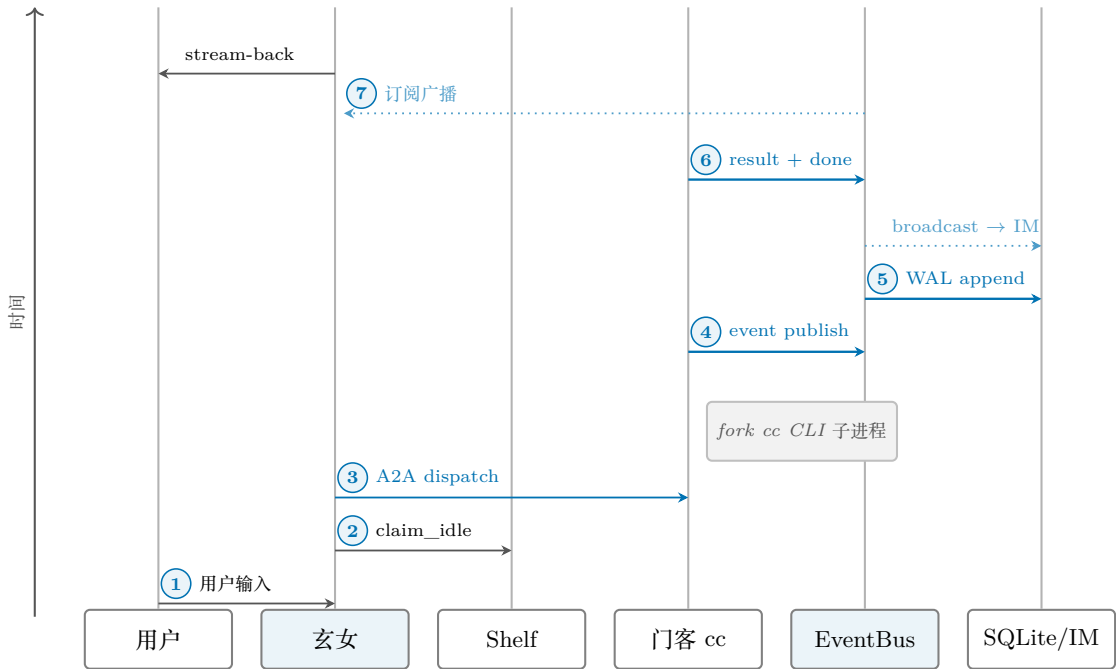


图 2.2 用户请求的端到端数据流（玄女接收 → dispatch → 门客 → EventBus → 多端扇出）

数据流分七步。第一步用户在 IM/PWA/TUI 端发出文本指令通过 Web-Socket 或 REST 抵达 fuxi-im 的 intervene handler。第二步 handler 调用 `Fuxi::intervene(xuannv_id, text)`，编排层先检查玄女当前 `ShelfStatus: Idle` 则退化为 `dispatch` 起一轮新 turn（避免 `send_message` 在无 active receiver 时被 drop），Busy 则把消息推入 `PendingOutbox` 等下次 turn 边界 drain。第三步玄女根据 system prompt 与上下文产出工具调用，或选择把子任务派给某个门客。第四步编排层 spawn 后台 pump task，把私有 receiver 的每个事件补上 `task_id/agent_id` 元数据再 `republish` 到公共总线。第五步 `EventBus::publish()` 同时投递到广播路径（broadcast 零拷贝扇出）与持久化路径（mpsc + WAL 串行落库）。第六步所有订阅者

从广播路径实时收到事件——Firehose 转发四端、IM 路由到对应任务客户端、记忆抽取器订阅 `DeliverableProduced` 触发候选事实抽取。第七步任务终态抵达后 pump 的 terminal 检测加 500 ms 缓冲窗口结束 republish，编排层把门客 `ShelfStatus` 摊回 Idle。

整条数据流有三处关键设计点。其一先订阅后查询的回放协议：当订阅者通过 `EventBus::replay(cursor, live_tail=true)` 请求从历史游标接收事件时必须先 `subscribe` 拿到 broadcast Receiver，再去 SQLite 拉历史，最后无缝拼接。反过来「先查历史再订阅」会留出窗口，期间产生的事件既未入历史快照也未被订阅者收到。其二 **lag** 哨兵与背压可观测性：当 writer 拥塞超阈值（默认 512 条），`publish()` 通过 broadcast 路径发出 Custom { "event_store_lagged", pending } 哨兵事件。哨兵自身有去重逻辑（同一拥塞窗口只发一次），不会反向加剧拥塞。这把背压从「不可见的死锁」变成「可见的告警」[14]。其三 `input-required` 在伏羲平台的端到端流转：当门客遇到歧义需要人工答复时，伏羲在 wire 上发出 `state: "input-required"`[19]，对应当前 A2A 规范 ProtoJSON 枚举 `TASK_STATE_INPUT_REQUIRED`。编排层把这一状态映射为 `ShelfStatus::AwaitingInput`，让整体调度器（包括跨节点 `NodeLoadProvider`）把「在等用户」的门客从「在工作」剥离。这条端到端流转把规范层语义提升为平台级可观测事件流转。

2.4 跨节点扩展

伏羲默认单节点完整运行，但通过显式扩展点支持多节点。当用户开启分布式模式时 `fuxi-cli` 在 daemon 启动阶段实例化 `DistController`，注入到编排层 `DistEnqueuer` 与 `NodeLoadProvider` 两个 trait 槽位。架构如图 2.3。

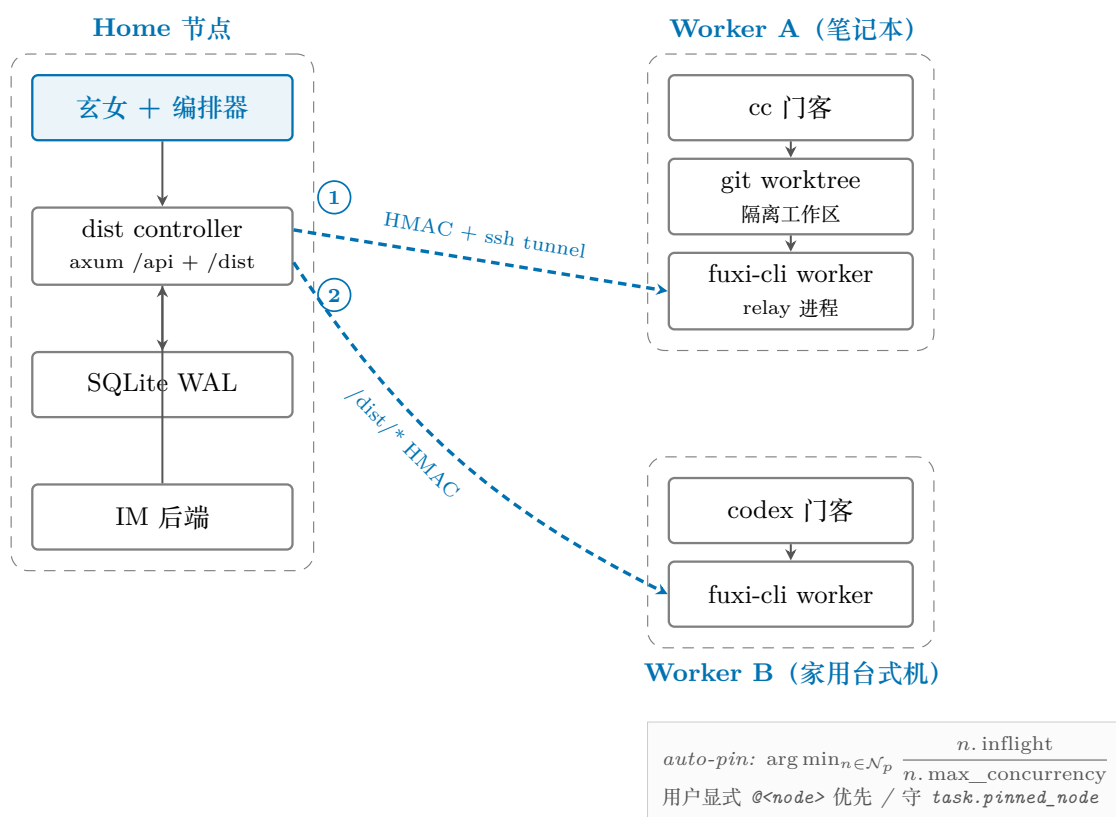


图 2.3 跨节点部署架构 (home 节点 + worker 节点 + 项目 host_nodes 路由)

跨节点核心数据是 Project 的 host_nodes 字段——每个项目可声明只能在哪些节点上跑。当任务关联到某项目时 `Fuxi::auto_pin_from_project` 读出 host_nodes，结合各节点当前负载选最合适节点。这条 auto-pin 遵循一条规约：当用户已显式 @<node> 钉死任务时，auto-pin 必须立即返回 None，调用方再加 `task.pinned_node.is_none()` 守卫双保险。这是「自动启发式 vs 用户显式意图」边界的明确化。负载选择函数为：

$$\text{node}(p) = \arg \min_{n \in \mathcal{N}_p} \frac{n. \text{inflight}}{\max(n. \text{max_concurrency}, 1)} \quad (2-1)$$

`max(·, 1)` 处理 `max_concurrency = 0` 边界。这与单节点门客选择函数形式相同——伏羲把同一套饱和度比较逻辑既用在节点层也用在门客层。

跨节点事件流通过分布式控制器双向打通：home 通过 request + JSON-RPC 把任务投递到 worker 的 `DistEnqueuer`；worker 产生的事件通过同一通道反向 republish 回 home 的 `EventBus`，所有跨节点事件在 `EventMeta.source_node_id` 字段打标。控制器是嵌入式的——它就是 `fuxi-cli` 中一个子模块，不需要独立运行时。未来若需更强一致性保障可在控制器内部接入 Raft[21]，但当前规模（家庭部署不超过 3 节点）下成本不划算。两条易被忽视的兼容性约束：Project 与 Task 所有新增字段必须 `#[serde(default)]`，否则旧版 `meta.json` 与旧 `events.db` 中的 task JSON 在 `registry.list()` 时会全部反序列化失败；worker 节点 pre-spawn 的 `git fetch` 必须 best-effort，失败只 log warn 不挂断 dispatch。

2.5 SLA 与性能预算

为让「高性能」从主观印象变成可度量约束，本节给出端到端任务派发延迟与 EventBus republish 滞后的形式化预算。记一次任务从用户输入到结果回传的端到端延迟为 L_{e2e} ，分解为四分量：

$$L_{e2e} = L_{enq} + L_{disp} + L_{exec} + L_{evt} \quad (2-2)$$

其中 L_{enq} 是任务入派发队列开销， L_{disp} 是编排层选门客并发派发指令开销， L_{exec} 是门客实际执行任务耗时（包括模型推理与工具调用）， L_{evt} 是事件从产生到抵达观测端的传播开销。在大多数实际工作负载中 L_{exec} 由模型推理时间主导（数秒至数十秒）。本设计为系统层分量设置 P99 预算： $L_{enq}^{P99} \leq 5 \text{ ms}$ 、 $L_{disp}^{P99} \leq 10 \text{ ms}$ 、 $L_{evt}^{P99} \leq 50 \text{ ms}$ 。 L_{exec} 因受模型推理影响不在系统层 SLA 范围。

事件总线 republish 滞后下界由式给出：

$$L_{evt} \geq \max_{i \in S} (T_{serialize} + T_{broadcast} + T_{consume,i}) \quad (2-3)$$

S 是订阅者集合， $T_{serialize}$ 是事件 JSON 序列化耗时， $T_{broadcast}$ 是 channel 内部分发耗时， $T_{consume,i}$ 是第 i 个订阅者消费耗时[14]。该不等式表明端到端延迟受最慢订阅者制约。伏羲的应对是：广播路径设置 lag 容忍阈值（默认 512 条），超过则丢弃旧事件并发出 `RecvError::Lagged`——业务侧全量历史走 replay 路径补齐，broadcast 不为慢消费者承担拖累快消费者的代价。

吞吐量与调度损耗形式化为：

$$T = \frac{N}{W_{total}}, \quad \eta = 1 - \frac{T}{T_{ideal}}, \quad T_{ideal} = \frac{K}{\tau} \quad (2-4)$$

N 是测量窗口内完成的任务数， W_{total} 是该窗口总耗时， K 是门客数， τ 是单任务理想耗时。 η 反映通讯链路、调度逻辑与同步开销在整体吞吐中的占比[20]。第四章 §4.2 吞吐实验以 $\eta < 0.15$ 作为 1~8 门客并发场景下的设计目标。公式 (2-2)~(2-4) 共同给出本文对「高性能」的可度量定义，第三章给每个分量在模块内部的具体优化，第四章给实测数据验证预算是否被满足。

2.6 本章小结

本章自顶向下给出了伏羲的总体设计：明确三组核心矛盾的解法并把功能/非功能需求与设计决策做成映射表；给出五层架构与横向支撑模块的整体布局，强调编排层与协议层正交分离让 A2A 保持轻量、观测层只读不写让多端订阅可以并存；沿一次完整用户请求揭示「先订阅后查询」「lag 哨兵」「input-required 端到端流转」三条关键设计点；给出跨节点扩展的钩子化设计与节点选择函数；给出端到端延迟分解、广播滞后下界与吞吐损耗三组形式化公式。下一章基于本章总体设计沿模块依赖自下而上展开核心模块的内部设计与实现。

3 系统设计与实现

第二章给出了伏羲的整体架构与通讯模型形式化。本章把这套架构落到 Rust 实现，按模块依赖自下而上的顺序展开七个核心单元的设计与实现：模块布局、事件总线、A2A 协议、编排与任务派发、执行适配器与三层沙箱、跨节点通讯、观测与横向支撑。每节遵循同一主线：先讲外部约束与关键数据结构、再用一段 Rust 代码片段或公式呈现工程上不寻常的部分、最后追加关键设计取舍。本章保留的代码片段均节选自主仓 /Users/e0_7/fuxi (HEAD: 04eb34c) 下对应文件，为版面计做了行宽折断与无关字段省略，未修改控制流。

为给整章提供一张索引图，图 3.1 把伏羲一次完整任务执行的端到端闭环画在一张图上：用户从 IM/PWA/REPL 输入指令（步骤 1），玄女接到 `intervene` 后向 Shelf 申领空闲门客（步骤 2），通过 A2A `tasks/sendSubscribe` 把任务发到门客的 JSON-RPC + SSE 端点（步骤 3），门客在 L2 git worktree 沙箱中执行（步骤 4）；执行过程中产生的所有事件向 EventBus publish（步骤 5），由 EventBus 同时走 broadcast 实时扇出（步骤 6a）与 WAL 串行落库（步骤 6b）；观测端断线重连时通过 `replay(cursor)` 从 SQLite 拉历史补齐（步骤 7），再无缝拼接到实时流；若门客进入 `InputRequired` 中间态，事件被翻译成 `ShelfStatus::AwaitingInput`（步骤 8），玄女据此请求人工回复（步骤 9）。后续小节展开的代码与算法均服务于这同一个闭环。

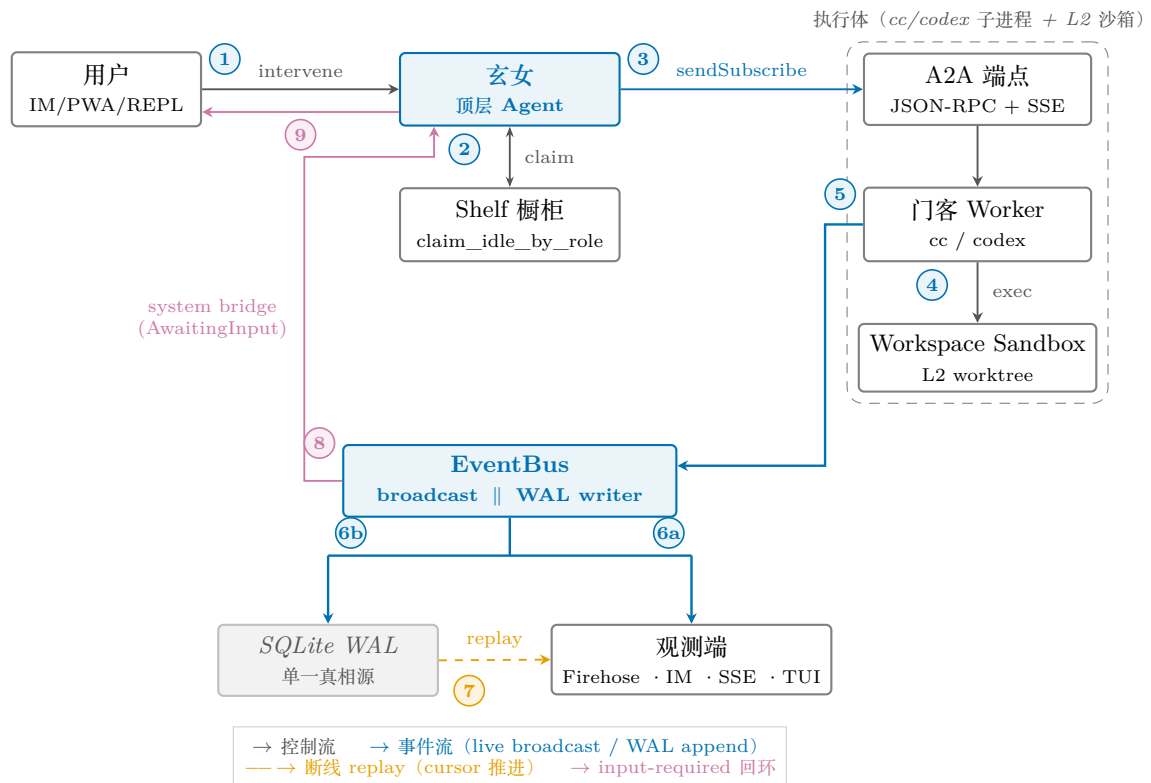


图 3.1 伏羲端到端任务执行闭环（9 步从用户提交至多端观测）

3.1 模块布局与 13 crate 边界

伏羲在 crate 粒度上划分 13 个模块，总代码规模 82515 行 Rust 源（`find crates -name "*.rs" | xargs wc -l`）。各 crate 按职责分层组织，依赖方向严格自下而上、禁止循环引用，整体如图 3.2。表 3.1 列出每个 crate 的代码规模、对外类型与主要职责。

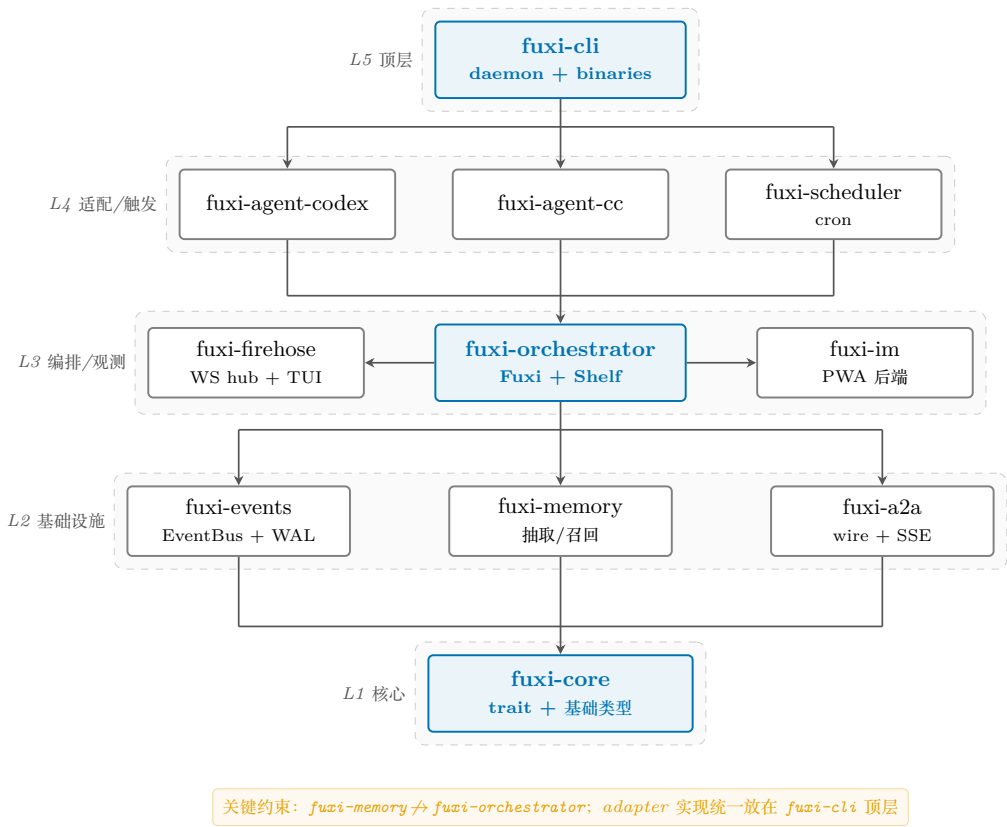


图 3.2 伏羲模块依赖关系（13 crate，5 层架构）

表 3.1 伏羲 workspace 13 个 crate 的边界与代码规模

Crate	LOC	对外类型/trait	主要职责
fxi-cli	33 408	二进制入口	REPL + daemon + dist controller + 钩子注入
fxi-im	16 149	axum app, REST/WS handlers	IM 后端：任务/对话/intervene 端点
fxi-orchestrator	10 220	Fuxi, Shelf, Recallsink	编排中枢：注册表 + dispatch + republish
fxi-agent-cc	3 900	CcAgent	Claude Code 适配器（WS 反连）
fxi-workspace	3 043	GitWorktree-Workspace	git worktree 隔离 + L1/L2/L3 沙箱
fxi-memory	2 643	OracleStore, ExtractorSpawner	长期记忆：事实/技能/身份
fxi-agent-codex	2 197	CodexAgent	Codex 适配器（spawn-per-dispatch）
fxi-scheduler	2 146	Keeper, Watcher	cron/once/webhook/fs 触发器
fxi-events	2 135	EventBus, EventStore	实时分发 + WAL 持久化 + 历史回放
fxi-firehose	2 135	Hub, axum routes	事件流四端输出（WS/SSE/REST/TUI）
fxi-core	2 102	Agent, Workspace, Event, Task	核心 trait 与值对象，零外部 IO
fxi-a2a	1 321	wire::*, Server, Client	A2A wire + axum 服务器 + request 客户端
fxi-skills	1 116	RoleLoader, SkillLedger	角色玉牒加载与招贤流程
总计	82 515		

fxi-cli 占 workspace 近 44%——这反映大型 Rust 项目的工程现实：协议、抽象与基础设施虽然概念上是「重头戏」，但把它们装配成一个能跑的产品需要大量胶水代码。模块边界设计的隐含原则是把会高频变化的「策略」与会长期稳定的「机制」分开放在不同 crate：fxi-core 中的 trait 与 EventKind 枚举属于稳定层，fxi-orchestrator 内部调度策略与 fxi-agent-cc 内事件翻译规则属于策略层，迭代频率高但不影响其他 crate。

依赖图中存在一处需绕过环路的需求：fxi-memory 在召回结果回传时需要往 orchestrator 持有的事件总线 publish 事件，但若让 memory 反向依赖 orchestrator 会形成循环。本设计采用「trait 在低层、impl 在高层」的反向依赖：在 orchestrator 中定义 RecallSink trait，由顶层 fxi-cli 提供实现并在 daemon 启动时注入；类似地有 ExtractorSpawner、DistEnqueuer、NodeLoadProvider 三个钩子，统一定义在低层 crate、由 fxi-cli 注入实现。这是规避 Rust workspace 循环依赖的标准 Robert Martin 依赖反转范式。

3.2 事件总线模块

事件总线由 fxi-events 实现，是公理「真实时不轮询」与「SQLite 是单一真相源」的工程落地。该模块同时承担三件事：实时分发、持久化、历史回放。一个 Event 由 Event-

Meta（携带通用元数据）与 EventKind（带 50+ 变体的 `#[serde(tag = "type")]` 标签联合）构成。`tag = "type"` 的关键价值是反序列化无歧义，且新增变体时所有 `match ev.kind` 处都会编译失败强制更新所有消费路径——这是 Rust enum 与 `serde` 在类型安全上的典型应用。

双路径架构。EventBus::publish() 在每次调用时同时把事件投递到两条独立路径：第一条是 Tokio broadcast channel[24] 零拷贝扇出给所有订阅者实现实时推送；第二条是异步 mpsc 队列由后台 writer task 串行落库到 SQLite，启用 WAL 模式[29] 让读写并发不互斥。两条路径各走最优形态，避免「单条通道既当广播又当持久化」的拥塞反模式。

非阻塞发布与 lag 哨兵。发布路径核心约束是「调用方永不阻塞」与「事件不丢」两个看似矛盾的约束，通过「try_send + 后台转交 + 哨兵告警」三步组合解决，见代码片段 1。

```
1 pub fn publish(&self, ev: Event) -> Result<()> {
2     // 路径 1: broadcast 零拷贝扇出，不影响 mpsc
3     let _ = self.inner.broadcast_tx.send(ev.clone());
4     // 路径 2: try_send 入 mpsc（非阻塞）
5     match self.inner.writer_tx.try_send(ev) {
6         Ok(_) => { self.inner.writer_pending.fetch_add(1,
7             Ordering::Relaxed); }
8         Err(TrySendError::Full(dropped)) => {
9             // 拥塞：后台 spawn 等慢车位，当前 publish 立即返回
10            let pending =
11                self.inner.writer_pending.load(Ordering::Relaxed);
12            if pending >= self.inner.cfg.lag_threshold {
13                self.maybe_emit_lag_sentinel(pending); // 哨兵告警
14            }
15            let tx = self.inner.writer_tx.clone();
16            tokio::spawn(async move { let _ =
17                tx.send(dropped).await; });
18        }
19        Err(TrySendError::Closed(_)) => return
20            Err(Error::WriterClosed),
21    }
22    Ok(())
23 }
```

Listing 1: EventBus::publish 非阻塞实现节选（fuxi-events/src/bus.rs）

maybe_emit_lag_sentinel 通过同一个 broadcast 路径发出一条 Custom { "event_store_lagged", pending } 哨兵事件，订阅者据此感知到底层 writer 已经堆了多少未落库事件。哨兵自身有去重逻辑，不会反向加剧拥塞。该设计与 Tower 在 Rust 服务栈中的 tower::load 背压抽象[30] 方向一致——都把背压视为一等公民信

号而非异常状态；NATS[31] 与 Kafka 同样把订阅者滞后纳入显式监控指标。替代方案有二：直接丢消息违反「事件不丢」公理；让 `publish()` 阻塞等待 mpsc 容量会让上层编排在偶发拥塞下卡死。两种简单方案均不可取。

回放与因果序。`EventBus::replay(cursor, live_tail)` 接受 `ReplayCursor` (`Beginning/FromId/FromTime`)，先从 SQLite 拉历史流，若 `live_tail = true` 则在历史末尾无缝拼接 broadcast 实时流。实现上必须先订阅 broadcast 再开始拉历史——反过来会留出窗口，期间产生的事件既未入历史快照也未被订阅者收到。事件因果序遵循 Lamport 逻辑时钟思路[14]：伏羲在每条事件 `meta` 字段记录 Agent ID、Task ID、墙钟时间戳与 SQLite `rowid`（单调递增），订阅者重建因果链时用 `rowid` 排序，避免引入向量时钟所需的时钟同步基础设施。

SQLite 工程细节。 事件持久化表 `events` 包含 `id/at/session/agent/task/kind_tag/payload` 七列，`kind_tag` 独立索引使按类型过滤为 $O(\log n)$ 扫描，`payload` 存放完整 Event JSON 保证 schema 演进时零破坏迁移。WAL 模式高并发下偶发 `SQLITE_BUSY`，伏羲通过 `busy_timeout=5s` 加三次线性退避（50/100/150 ms）封装在 `EventStore::append` 内部，外层调用方无感知。`:memory:` 库每条连接独立持有内存库，`EventStore::new` 检测到时强制 `max_connections = 1` 规避「明明 `append` 了却 `replay` 不出来」的诡异行为。整个 `fuxi-events` crate 共 1673 行（`store.rs` 1024 + `bus.rs` 649），是从外观简单的 `publish/subscribe/replay` 接口背后挖出工程取舍最多的模块之一。

3.3 A2A 协议模块（论文核心贡献）

通讯协议模块由 `fuxi-a2a` 实现，是本论文最重要的工程贡献之一。Rust 生态已有 `a2a-rs`（A2A 项目官方 SDK）以及 `a2a-client`、`a2a-types` 等第三方 crate[19]，本工作从零实现 wire 层、JSON-RPC 信封、SSE 流抽象、axum 服务器与 request 客户端，共计 1063 行核心代码加 258 行端到端测试，与 `fuxi-events` 和 `fuxi-orchestrator` 同源演进。

协议层次划分。A2A 协议解决「Agent 与 Agent 之间的协作契约」，MCP[18] 解决「模型与外部工具的调用规范」，二者面向不同语义层次不互斥。伏羲内部门客之间使用 A2A 传递任务与结果，门客调用外部工具沿用其底层 CLI 自身的工具机制（cc 与 codex 都遵循 MCP 风格的工具声明）。这一分工与 SWE-agent[8] 的 Agent-Computer Interface 思想一致。替代方案曾考虑把所有协议统一到 MCP 让门客之间通过虚拟工具相互调用，被否决的原因是 MCP 工具语义偏「无状态、单次调用」而 Agent 协作需要「状态机、流式响应、人工介入点」。这里需进一步区分：A2A 描述「Agent 之间应当传递什么」，事件总线描述「系统内部已经发生了什么」——前者是动作语义，后者是事实语义，伏羲明确划清这条边界让 A2A 信封只承担最小契约。

Crate 模块拆分。 `crates/fuxi-a2a` 拆分 7 个文件：`wire`（458 行）、`jsonrpc`（77 行）、`sse`（45 行）、`server`（196 行）、`client`（203 行）、`error`（52 行）、`lib.rs`（32 行），加端到端测试 `tests/roundtrip.rs` 258 行，总计 1321 行（文档注释占 25%）。本实现采用早期 A2A JSON-RPC binding，沿用方法命名 `agent/getCard`、`tasks/send`、`tasks/sendSubscribe`、`tasks/get`、`tasks/cancel`（当前官方

规范已切到 PascalCase 方法集 SendMessage、GetTask 等，本工作未跟进改造，亦未与 a2a-rs 做完整 wire-level 互通验证；不主张「v1.0 兼容」。

wire 层标签联合。核心数据结构包括 AgentCard、Task、Message、Part、Artifact。所有结构使用 serde 派生序列化，并通过 `#[serde(rename_all = "kebab-case")]` 与 `#[serde(tag = "type")]` 精确控制 wire 上 JSON 形态。代码 2 是 Part 的写法。

```
1 #[derive(Debug, Clone, Serialize, Deserialize)]
2 #[serde(tag = "type", rename_all = "kebab-case")]
3 pub enum Part {
4     Text { text: String },
5     Data { data: serde_json::Value },
6     File { file: FileContent },
7 }
```

Listing 2: Part 类型的 serde 标签联合实现

使用 `tag = "type"` 而非 `untagged` 是经过权衡的——`untagged` 让 `serde` 按字段形态依次尝试匹配，在 `Text` 与 `Data` 字段近似时会产生反序列化优先级歧义；显式标签强制每个 Part 在 wire 上携带 `"type": "text"` 类判别字段，反序列化结果稳定可预测。`fuxi-core::AgentCard`（带 `id`、`status` 运行时字段）与 `fuxi-a2a::wire::AgentCard`（协议对外视图）之间的边界转换通过 `From trait` 实现：丢弃内部字段、把 `system_prompt` 首行截 200 字作 `description`、把 `tags` 平铺成 `A2A skills`。本设计禁止在任何业务路径中手抄字段——所有边界转换必须走 `From trait`，由编译器保证两种 `AgentCard` 同步演进。

InputRequired: A2A 规范语义在伏羲平台的映射。A2A 当前规范定义的 `TaskState` 包含 `Submitted`、`Working` 两个进行态，`Completed`、`Failed`、`Canceled` 三个终态，以及 `InputRequired`、`AuthRequired` 两个 interrupted state（非终态、可恢复回 `Working`）和 `Rejected` 这一拒绝执行终态。伏羲提前实现 `InputRequired`（对应规范后续正式纳入的 ProtoJSON 枚举 `TASK_STATE_INPUT_REQUIRED`）；`AuthRequired` 与 `Rejected` 当前实现暂未覆盖（详见 §5.2）。代码 3 是 `fuxi-a2a` 当前枚举。

```
1 #[derive(Debug, Clone, Copy, PartialEq, Eq, Serialize, Deserialize)]
2 #[serde(rename_all = "kebab-case")]
3 pub enum TaskState {
4     Submitted,           // 刚入队
5     Working,             // 处理中
6     InputRequired,       // 需要人工输入（早期 draft 未列、fuxi 提前实现）
7     Completed, Failed, Canceled, // 三个终态
8 }
```

Listing 3: fuxi-a2a TaskState 枚举（提前实现 InputRequired 中间态）

这一平台映射的工程价值在于把人工介入从「编排器内部隐式状态」提升为「可被协议字段承载、可被事件流观测的显式状态」。玄女订阅门客的 SSE 流时一旦看到 `state: input-required` 可立即把对话权切换回用户，等用户回复后再走 `tasks/send` 把答复回传给同一 Task ID。在伏羲实际使用中这一机制覆盖「门客对需求理解有歧义」「门客对工具调用授权犹豫」「门客遇到外部依赖问题」三类场景，全部通过同一状态语义统一处理。`InputRequired` 与终态的关键区别是：终态不可恢复，事件流关闭；`InputRequired` 是中间态，事件流保持打开，等用户回复后转回 `Working`。这条「中间态可恢复」让多轮对话与多回合协作能在同一 Task 上下文内完成。进一步地 `InputRequired` 与编排层 `ShelfStatus::AwaitingInput` 一一对应——使整体调度器（包括跨节点 `NodeLoadProvider`）能将「在等用户」的门客负载与「在工作」的负载区分开。

单端点 `HTTP+SSE` 与帧解析。伏羲只暴露一个 HTTP 端点 `POST /a2a`，所有方法通过 JSON-RPC 2.0 信封 `method` 字段分发；`tasks/sendSubscribe` 是流式方法，对它的响应被升级为 `text/event-stream`[28] 在同一 TCP 连接上回传任务进展。这一设计避免客户端管理两条独立连接的同步问题。SSE 帧的客户端解析需处理 TCP chunk 边界上的不完整帧，伏羲的 `parse_sse_stream` 用增量缓冲在每次收到字节后查找 `\n\n` 分隔符，已确证能在跨包边界场景下正确切帧（算法 4）。

```
1 Input:  byte_stream
2 Output: 流式产出 ServerSentEventPayload
3   1: buf <- ""
4   2: while chunk <- byte_stream.next() do
5     3:   append chunk (utf8) to buf
6     4:   while buf 含 "\n\n" 边界 do
7       5:     (frame, rest) <- buf.split_at("\n\n")
8       6:     buf <- rest
9       7:     payload <- decode_frame(frame)    // 多行 data: 拼接, :
          注释跳过
10    8:     yield payload
11    9:   end while
12 10: end while
```

Listing 4: A2A 客户端 SSE 帧解析（`fuxi-a2a/src/client.rs::parse_sse_stream`）

性能上选 `String` 增量缓冲 + `find("\n\n")` 朴素策略而非 `tokio_util::codec::FramedRead` 工业级 codec——SSE 帧体积通常很小（`status < 4KB`、`artifact < 64KB`），单帧重扫开销可忽略，codec 路线代码量翻倍但收益微乎其微。`tests/roundtrip.rs` 包含三组关键测试：`roundtrip_all_methods` 跑完 5 个 RPC 方法的完整调用闭环；`roundtrip_subscribe_stream` 在 server 端故意把 SSE 帧切成 1 字节 chunks 发出验证最恶劣 chunk 边界下的正确性；`unknown_method_returns_jsonrpc_error` 验证 server 对未知方法返回 `200 + -32601 error` 而非 `HTTP 4xx`——这条 invariant 是 JSON-RPC 规范要求。

3.4 编排与任务派发模块

编排模块由 `fuxi-orchestrator` 实现是整个系统的中枢，核心职责包括：维护门客注册表与生命周期、把任务派给合适门客、提供事件 `republish` 通路。该模块代码量约 8039 行。

玄女一门客分层。伏羲采用「玄女顶层 Agent + 门客 worker Agent」分层结构，与 MetaGPT[12]「角色分工 + 显式协作」思想一致但更强调编排层不可绕过——所有门客之间通讯必须经玄女知情，任何调度决策都发出对应事件。这条公理在 `Fuxi::shutdown_agent` 中反复实施：所有 `shutdown` 路径必须比对玄女 ID，命中则 `warn` 并 `noop` 返回，唯一例外是显式 `shutdown_xuannv_for_handoff`——该命名本身即作为约束，使 `grep` 即可识别「不要在别处调用」的意图。注册表 `Shelf` 用 `Arc<RwLock<HashMap<AgentId, ShelfEntry>>>` 实现，工作模式是「大量读 + 少量写」符合 `RwLock` 偏向读优化语义。

Worker 选择函数。给定任务 t 与门客集合 $\mathcal{W}$ ，伏羲调度函数选择满足约束的最低负载者：

$$\text{select}(t) = \arg \min_{w \in \mathcal{W}} \left\{ \frac{w.\text{inflight}}{w.\text{max_concurrency}} \mid \text{tags}(t) \subseteq \text{tags}(w), \text{pin}(t) \in \{\emptyset, w.\text{node}\} \right\} \quad (3-1)$$

三层过滤：标签兼容性、节点钉死约束、最低饱和度。`max_concurrency = 0` 边界归一化为 1 避免除零。这套规则与 MapReduce[32]「数据本地性 + 负载均衡」思路一致但更轻量。`pinned_node` 优先级守卫：当用户在 IM 端显式 `@<node>` 钉死任务时 `NodeLoadProvider::auto_pin_from_project` 必须立即返回 `None`——不能再悄悄改写 `pinned_node`；调用方再加 `task.pinned_node.is_none()` 判断作为双保险。这是分布式自动负载均衡与用户显式意图之间的边界明确化。

原子 **claim** 与 **dispatch pump**。早期版本将 `find_idle_by_role` 与 `set_status(Busy)` 拆为两步，并发 `dispatch` 可能 `find` 到同一只门客并分别 `set_status`，使同一门客被双派；现在 `claim_idle_by_role` 在 `write().await` 锁内完成「找空闲 + 置 `Busy` + 清 `idle_since`」三步原子操作（代码 5）。第二个并发陷阱是 `idle GC`：另一条独立线程会扫 `idle_since` 字段超阈值的门客并 `shutdown_agent`，若 `claim` 时不清掉 `idle_since`，已 `busy` 的门客仍被 `GC` 视为长时间空闲而误杀；显式置 `None` 才能让 `GC` 与 `claim` 不冲突。

```
1 pub async fn claim_idle_by_role(&self, role: &str) ->
  Option<AgentId> {
2   let mut guard = self.inner.write().await;
3   let pick = guard.iter_mut()
4     .find(|(_, e)| e.status == ShelfStatus::Idle &&
5       e.card.profile.role == role);
6   if let Some((id, e)) = pick {
7     e.status = ShelfStatus::Busy;
8     e.idle_since = None; // 占用必须清，否则 GC 误杀
    Some(*id)
```

```

9     } else { None }
10 }

```

Listing 5: 原子按角色 claim 一个空闲门客 (fuxi-orchestrator/src/registry.rs)

Fuxi::dispatch 主循环见算法 6 (隐去 v2 分布式分支细节, 保留控制流主干)。

```

1 Input: agent_id, task
2 1: if task.pinned_node is None and task is in a project then
3 2:     task.pinned_node <- auto_pin_from_project(task)    //
      saturation 最低
4 3: end if
5 4: if task.pinned_node is Some or task.required_tags is non-empty
      then
6 5:     emit TaskCreated; call dist_enqueuer.enqueue(task); return
      // 远端接管
7 6: end if
8 7: agent <- shelf.get_agent(agent_id)
9 8: rx <- agent.dispatch(task)                            // 拿门客私有 mpSC
      Receiver
10 9: spawn pump task:
11 10:     saw_terminal <- false
12 11:     loop:
13 12:         ev <- rx.recv() with timeout 50ms
14 13:         if ev is Some(e) then
15 14:             e.meta.task <- task.id; publish e to EventBus
16 15:             if e.kind is terminal then saw_terminal <- true
17 16:             elif rx is closed then break
18 17:             elif saw_terminal then break    // grace 超时, 安全退出
19 18:             end if
20 19:         end loop

```

Listing 6: 本地任务派发主循环 (fuxi-orchestrator/src/fuxi.rs::dispatch)

第 12 行 50ms grace 窗口是实测教训: cc 在发出 ResultSuccess 后还会异步 flush 几条 DeliverableProduced 事件, 立即 break 会让它们被丢; M2.1 修复后给 50ms 缓冲再退全部能 republish。auto_pin_from_project 是 v2 跨节点版本加入的: 调 NodeLoadProvider 取每个候选节点 saturation = inflight / max_concurrency 选最低, 候选全离线时返 None 退化到本地 spawn。

intervene 退化语义与 SystemEventBridge。当用户在玄女对话中插入新指令时, 玄女若处于 Idle 状态没有活跃 active_tx, 直接 send_message 会让消息悬空。修复思路是 Fuxi::intervene 入口先查 ShelfStatus: Idle 退化为 dispatch 起一轮新 turn, Busy 把消息推入 PendingOutbox 等下次 turn 边界 drain。这条「Idle 退化为 dispatch」规则避免了「玄女一忙就丢消息」的常见症状。SystemEventBridge 是独立

tokio task: 启动后立即 `bus.subscribe()` 拿 broadcast Receiver, 循环读取每条事件, 对 `AgentDead/TriggerFired/TaskStateChanged` 等系统事件做翻译, 转成「系统提示文本」通过 `Intervener::intervene_system_origin` 喂给玄女。Lagged 错误被静默 filter 保证单次落帧不杀整条订阅链。这条桥是公理「真实时不轮询」的具体落地——玄女完全不查询任何状态, 所有「知情权」都通过 EventBus 推送实现。

3.5 执行适配器与三层沙箱

执行层把抽象 Agent trait 落地到具体 CLI 进程上。伏羲实现两个适配器 (`fuxi-agent-cc`、`fuxi-agent-codex`), 通过 `fuxi-workspace` 三层沙箱保证多门客并发文件隔离。

cc 与 codex 适配器差异。CcAgent 包装 Claude Code CLI, 采用「长连接 WS + 多轮 dispatch」: 进程启动一次后通过 WebSocket 反连, 多次 dispatch 复用同一进程。Codex-Agent 包装 Codex CLI, 采用「spawn-per-dispatch」: 每次 dispatch 起一新进程, prompt 作为位置参数传入, 进程结束即销毁。差异源自 CLI 自身能力——cc 支持 SDK URL 与会话保持, codex 是 one-shot exec 模式无 stdin 输入。CodexAgent::send_message 直接返回错误, 意图是强制上层把「对 idle codex 门客发消息」退化为重新 spawn 一个新进程把消息作为新 prompt 喂入——用户视角无感。

WS 反连关键工程细节。见代码 7。launch_with_id 强制传入 id 参数——曾踩过的坑是编排层和适配器各自生成 AgentId, 导致 AgentSpawning 与 AgentReady 事件挂在不同 ID 上订阅者无法匹配。这条新规约是「新 agent 适配器三铁律」之一: 必须接收外部传入的 ID, 绝不自己生成; 事件 pump 无论怎么退出都要把 ShelfStatus 摊回 Idle; find-then-act 类接口必须原子 (如 claim_idle_by_role)。

```
1 pub async fn launch_with_id(id: AgentId, profile: AgentProfile, cfg:
   CcLaunchConfig) -> Result<Self> {
2     let channel =
        Arc::new(WsChannel::bind(&Uuid::new_v4().to_string()).await?);
3     let mut child = spawn_claude(&cfg)?;           // 注入 --sdk-url
        + NO_PROXY (避开 Clash TUN)
4     tokio::select! {
5         Ok(_) = channel.wait_connect(CONNECT_TIMEOUT) => {}
6         Ok(_) = child.wait() => return
        Err(CcError::EarlyExit.into()),
7     }
8     let pump = tokio::spawn(pump_loop(channel.clone(), /* ... */));
9     Ok(CcAgent { card: AgentCard { id, profile, endpoint:
        channel.url(), .. },
10                inner: Arc::new(Mutex::new(Inner { channel, child,
        .. })), _pump: pump })
11 }
```

Listing 7: CcAgent::launch_with_id 反连握手节选 (fuxi-agent-cc/src/agent.rs)

三层沙箱 L1/L2/L3。L1ReadOnly 是项目源码与依赖的只读视图所有门客共享；L2Ephemeral 是任务级临时 worktree，每次 dispatch 创建独立分支、任务结束自动清理；L3Persistent 是角色级持久沙箱，同一角色（如 dev-frontend）跨任务复用同一目录与 build cache。三层中 L2 最关键，落地依赖 git worktree[26]：git worktree add -b agent/<id> <path> <base> 在工作目录之外创建独立工作树与分支，多个 worktree 文件互不影响、共享同一 .git 目录节省磁盘。L3 持久沙箱面向构建缓存复用：编译型语言的 target/、node_modules/ 在每次任务结束时清理代价巨大，L3 让同一角色跨任务复用把 cargo build 从冷启动几分钟降到几秒。L1 只读层面向「门客只读取项目源码」轻量场景，跳过 worktree 创建直接挂载，启动延迟从百毫秒级压到十毫秒级。

进程边界 env 清理。子进程继承父进程环境变量是 Unix 默认行为，但在 cc/codex 这类「自身就是 cc agent」的 CLI 中会触发嵌套检测——父 cc 起子 cc 时子进程看到 CLAUDECODE 等 env 会静默退出。spawn_claude 在启动前显式 env_remove 一组前缀（CLAUDECODE、CLAUDE_CODE_*、CODEX_*），同时注入 NO_PROXY=127.0.0.1,localhost 规避用户本地 Clash TUN 代理把 WebSocket 反连吞掉的常见环境陷阱。

3.6 跨节点通讯实现

伏羲 v2 引入跨节点能力：home 机可把任务分发到 worker 节点执行。这套分布式层借鉴 ZooKeeper[22]「会话 + watcher」心跳模型，但实现做大幅简化——伏羲面向「个人开发者多机部署」不是数据中心规模，没有引入 Raft[21] 共识协议，而是接受「home 节点是单点协调者」简化模型。Raft 等协议需至少三节点 quorum 方可保证可用性；个人开发者通常只有一台 mac mini 加一台公司笔记本共两节点，启用 Raft 反因 quorum 不足而不可用。

dist controller 是部署在 home 节点的 axum service，监听同一进程内两套路由：/api/* 走用户 cookie 鉴权（IM 端使用），/dist/* 走 HMAC-SHA256 鉴权（worker 节点使用）。HMAC 签名按全要素 canonical bytes 拼接生成：

$$\sigma = \text{HMAC-SHA256}(K, \text{method} \parallel \text{path} \parallel \text{ts} \parallel \text{nonce} \parallel \text{body}) \quad (3-2)$$

K 通过 FUXI_DIST_HMAC_SECRET 环境变量注入，五要素覆盖请求方法、路径、毫秒时间戳、随机 nonce 与请求体；验签端按相同公式重算并做常量时间比较（算法 8）。

```
1: if |now - ts| > max_skew_ms then return ClockSkew // 同时防
    replay 与 future-replay
2: expected <- HMAC_SHA256(K, method || path || ts || nonce || body)
3: if length(expected) != length(provided_sig) then return
    BadSignature
4: if not constant_time_eq(expected, provided_sig) then return
    BadSignature // 防 timing oracle
5: if not NonceCache.check_and_insert(nonce) then return
    ReplayedNonce // 签名验过再查 nonce
```

Listing 8: 分布式请求 HMAC-SHA256 验签 (fuxi-cli/src/dist_auth.rs)

三个安全要点：时间戳偏差检查放最前同时防 replay 与未来时间戳预制；常量时间比较用 `subtle::ConstantTimeEq::ct_eq` 防 timing oracle；nonce 检查刻意放在签名验证之后，防止攻击者用大量伪造请求灌满 nonce cache 实现 DoS。worker 节点与 home 之间四条路径：`/dist/register` (worker 上线声明能力)、`/dist/heartbeat` (每 2 秒上报心跳与 inflight 计数)、`/dist/pull` (worker 主动来 home 取活)、`/dist/event` (worker 把事件回传 home)。设计上保持 stateless：home 不主动推任务给 worker 而是 worker 周期性 pull——这样 worker 在防火墙后也能工作无需暴露公网端口。

worker 节点 inflight 回收靠 `WorkerHeartbeat` 与 `WorkerStaleSwept` 两个事件实现。home 端订阅 EventBus 的 watcher 维护 `Map<NodeId, last_seen>`，超过 30 秒未收心跳的 worker 被标记 stale，其上 inflight 任务全部 republish 为 `TaskFailed{cause: "worker_stale" }`，让玄女决定是否重试。30 秒在 2 秒心跳间隔下相当于允许丢 14 次心跳——过短会让用户在地铁上断网 5 秒就丢任务，过长会让真正死掉的 worker 长时间占着 inflight 槽。worker 端 `spawn_worker` 时 `git fetch` 是 best-effort：失败只 warn 不 fail，避免短暂网络抖动让任务凭空消失。

3.7 观测、IM 与横向支撑模块

`fuxi-firehose` 内部 Hub 持有 EventBus 引用，对外提供四个端点：`/ws` 给 TUI 与 CLI 工具消费，`/sse` 给浏览器与 `curl` 用户消费，`/events` 是 REST JSON 历史查询 (不做 live tail，回避公理 #3 反例)，还有进程内 broadcast channel 给同进程组件零拷贝消费。同一份事件经四种形态出口、共用底层 `EventBus::replay` 接口、handler 自身只负责协议翻译——这种「内核统一、出口多样」结构让新增协议 (如 gRPC streaming) 只需再加一个二三十行 handler。同时支持 WS 与 SSE 两种协议而不强制统一的理由是：TUI 端需要双向 (接收事件 + 上报取消信号) WebSocket 自然适配；浏览器与 `curl` 用户更适合单向 SSE 无需手动管理 ping/pong。

`fuxi-im` 后端面向 PWA 与桌面 IM 客户端，提供 REST 任务管理 (`GET /api/tasks`、`POST /api/tasks/<id>/intervene`)、对话历史拉取、双向消息流 (WS `/ws/messages`) 与设备推送注册。后端业务调用直接走 `Arc<Fuxi>` 句柄——`intervene` 端点直接调 `Fuxi::intervene`，由编排层判断 idle 退化 dispatch 还是 busy 入 pending 队列，IM 层不重复实现这套语义。`fuxi-im` 在路由装配上采用「skeleton 稳定、实现可填」分工：`router.rs` 中所有 handler 签名一次定型，路由装配集中在 `build()`；具体业务实现按 owner 分布在 `handlers/*.rs`。

横向支撑三件套。`fuxi-memory` 基于 SQLite 与 FTS5 全文索引维护三张核心表：`oracle_facts` (甲骨：事实三元组)、`hetu_patterns` (河图洛书：角色技能样本)、`user_profiles` (身份卡)。三张表写入策略统一为「只 ADD 不 UPDATE」——冲突时不覆盖旧行而是把旧行 `valid_until` 标 `now()` 再 INSERT 新行，保证完整审计追溯。FTS5 用 `unicode61` 分词器，实测 1 万条事实下 p95 查询小于 10 毫秒。`fuxi-skills` 是角色玉牒加载器，每个角色以 `roles/<role>/ROLE.md` 存储，借鉴 Voyager[7] 持久

技能库思路并增加 staging + ledger 双轨——新角色先写入 roles/<role>.staging/ 经审阅后才 mv 到 roles/。fuxi-scheduler 提供 cron/once/webhook/FileWatch 四类触发器，统一通过发出 TriggerFired 事件接入主链——这条间接路径让 scheduler 自身不需要持有 Fuxi 句柄避免又一处潜在循环依赖。fuxi-memory 的「自动事实抽取」默认关闭，需用户在配置中显式打开——这是因为早期实测发现抽取器在用户与玄女闲聊时容易把无关闲谈写进事实库污染长期记忆；打开后抽取器作为独立子门客被 spawn（进程隔离）从指定 task transcript 中筛候选事实，写库前还会过 confidence 阈值过滤（默认 0.7）。

3.8 本章小结

本章把第二章的设计落到代码：模块布局确立 13 crate 边界与「trait 在低层、impl 在高层」反向依赖；事件总线用「try_send + 后台转交 + 哨兵」三步组合调和实时性与持久化矛盾；A2A 协议从零实现 wire 层、JSON-RPC 信封、SSE 帧解析，并把 InputRequired 中间态在伏羲平台层映射为人工介入信号；编排层用 publish-then-pump 模式把门客私有 receiver 转换为全局事件流，并通过原子 claim_idle_by_role 与 shutdown_agent 玄女豁免规避并发陷阱；执行层用 WS 反连打破传统 stdio 范式让编排层主动控制门客，三层沙箱让多门客并发文件操作互不干扰；分布式层用 HMAC-SHA256 + nonce cache + 常量时间比较保证跨节点通讯安全；观测与 IM 层把所有事件向多种端形态扇出。下一章在这套实现之上做端到端实测——延迟、吞吐、错误恢复、跨节点协作场景的量化数据。

4 系统测试与结果分析

第三章给出了伏羲（Fuxi）的设计与实现细节，本章在该实现基础上对系统进行多维度测试。测试目标分为正确性、实时性、吞吐扩展性、参数敏感性以及通讯层压力承受能力五大类，覆盖从底层事件总线到顶层任务编排的完整链路。所有定量实验在同一台机器完成，配置：Apple M4 Pro（10 P 核 + 4 E 核 / 14 物理核）、48 GB 统一内存、macOS 26.0.1、Rust 1.93.1 stable、SQLite 3.51.0、Tokio 多线程运行时显式 worker_threads(8)。吞吐采用 5-run median 抑制冷启动，延迟采用 500 独立样本统计 p50/p99，事件总线压测每 cell 5 秒稳态。完整原始数据见仓库 deliverables/thesis-v2/benchmarks/v2-2026-05-07.md。

4.1 测试目标与方法学

正确性通过 Rust 单元测试与跨 crate 集成测试验证；实时性通过任务派发与事件总线广播的端到端延迟测量验证，对应分布式系统因果序传递的经典指标^[14]；吞吐扩展性通过固定任务粒度、扫 Worker 数量的水平扩展实验验证，思路与 MapReduce^[32]一致；参数敏感性通过固定其他变量、扫描 poll_ms 的消融实验验证；通讯层压力承受能力通过事件总线纯压测验证，参考了 Kafka 等持久化消息系统的纯通讯极限测试^[20]。性能测试以独立 cargo bench 二进制 release profile 编译，所有数据由 bench 写入 markdown 报告，论文中所有图表由 matplotlib 脚本基于该报告自动生成，规避「论文里数字与仓库里数字不一致」常见问题。

测量方法学方面采用以下控制手段：吞吐与扩展性实验「5 次重复取中位数」抑制冷启动，对应顶刊 systems 论文常见做法；延迟实验在采集 500 样本前先丢弃 50 个 warmup；测试期间关闭 Spotlight 索引、Time Machine、IDE 索引等可能消耗 CPU 的进程并选夜间安静时段；Tokio 多线程运行时显式 `worker_threads(8)` 固定避免不同实验线程数变化导致结果不可比；时钟使用 `std::time::Instant`（macOS 上为 `mach_absolute_time`，分辨率约 1 ns），但延迟统计经 `SystemTime` 的 ns→us 整数除法截断，凡 p50 报告为 0.0 us 处实际含义为「< 1 us」。基于 500 个独立样本，p99 估计的 95% Wilson 置信区间约 ±2%，远小于本章关注的数量级差异。

baseline 选择上，本章未引入外部系统作为实测对照——公开的 AI Agent 编排系统多以 Python 实现，跨语言、跨抽象层级的 baseline 容易让数字失去工程意义（差异主要来自语言运行时与同步异步模型，而非编排层设计取舍），且容易让读者形成「Rust 比 Python 快」的浅层结论。本章采用理论上限对照（每 cell 直接给出 `tasks_n × 1000 / job_sleep_ms` 作为零开销完美调度上限），使读者可直接读出 Fuxi 与这一理想上限之间的工程损耗。

4.2 测试用例覆盖

Fuxi 的集成测试在每一个 crate 中按其对外暴露的能力组织，完整覆盖如表 4.1。总计 27 个集成测试文件分布在 13 个 crate，覆盖通讯、编排、执行、观测与支撑模块的全部主要路径。

表 4.1 系统测试覆盖情况

模块	主要测试文件	测试目标
fuxi-a2a	roundtrip.rs	A2A wire 编解码与往返一致性
fuxi-events	integration.rs	事件发布、订阅与回放
fuxi-agent-cc	fixture_stream.rs、 real_cc_smoke.rs	Claude Code 输出解析与真实启动
fuxi-agent-codex	fixture_stream.rs、 real_codex_smoke.rs	Codex 输出解析与真实启动
fuxi-cli	chaos_resilience.rs 等 5 文件	CLI、网关重启、分布式与召贤 e2e
fuxi-im	router_smoke.rs 等 3 文件	IM API、WebSocket 与认证
fuxi-firehose	hub_roundtrip.rs	Firehose 事件 hub 往返
fuxi-memory	oracle_roundtrip.rs 等 4 文件	长期记忆、可复用模式与抽取
fuxi-orchestrator	dispatch.rs、deliver- able_handoff.rs	任务派发与交付物交接
fuxi-scheduler	e2e_scheduler.rs	定时触发链路
fuxi-skills	loader.rs 等 4 文件	角色加载、暂存、登记与模板
fuxi-workspace	integration.rs	工作区创建与隔离

集成测试覆盖广度并不等价于行覆盖率指标——Rust 异步生态下 `cargo-llvm-cov` 对 `tokio::select!`、`tokio::spawn` 内部分支的归因尚不完整，强行追求百分数易诱导写「为覆盖而覆盖」的低价值测试。本章选择以「关键路径必有 e2e」「关键 wire 必有 round-trip」「关键 FSM 必有状态迁移枚举」三项硬规则替代覆盖率门槛。最

近一次提交中 `cargo test --workspace --all-targets` 完整通过，IMPWA 前端 `pnpm test`、`typecheck` 与 `lint` 也已通过。集成测试之外还使用 property-based testing (`proptest` 对 wire round-trip 不变量验证)、回归 fixture 沉淀（每次发现新 bug 优先沉淀一条 fixture）与 smoke 测试（`smoke-p0.sh` 汇总系统启动—对话—门客派发—回收的完整链路）三类辅助验证手段。

4.3 吞吐量、扩展性与端到端延迟分解

吞吐量测试基于第二章 SLA 节给出的端到端延迟分解 $L_{e2e} = L_{enq} + L_{disp} + L_{exec} + L_{evt}$ 与吞吐损耗 $\eta = 1 - T/T_{ideal}$ 。本节把通讯与调度相关的三个分量合记 $T_{comm} \equiv L_{enq} + L_{disp} + L_{evt}$ 、把任务执行单独写作 $T_{exec} \equiv L_{exec}$ 。基线吞吐结果如表 4.2。

表 4.2 分布式任务吞吐量基线测试结果 (5-run median)

worker_n	job_sleep	tasks_n	median_wall_ms	tasks/s	理论上限	Fuxi 损耗
1	10 ms	100	1227	81.50	100.00	18.5%
1	100 ms	50	5123	9.76	10.00	2.4%
1	1000 ms	10	10040	1.00	1.00	0.4%
4	10 ms	400	1212	330.03	400.00	17.5%
4	100 ms	200	5110	39.14	40.00	2.2%
4	1000 ms	40	10033	3.99	4.00	0.3%
8	10 ms	800	1203	665.00	800.00	16.9%
8	100 ms	400	5124	78.06	80.00	2.4%
8	1000 ms	80	10046	7.96	8.00	0.5%

表中三组层次分明的现象：10 ms 任务下损耗稳定在 17%–19%，100 ms 损耗骤降到 2.2%–2.4%，1000 ms 损耗降至 0.3%–0.5%。三段递减的物理原因是 T_{comm} （通讯与调度开销）在不同任务粒度下的占比差异： T_{comm} 包含 axum HTTP 往返（约 4–6 ms）、Tokio 任务调度（约 2–3 ms）、SQLite 状态写入（约 1–2 ms）以及 Worker 拉取轮询（`poll_ms` 平均一半）。具体地，10 ms 任务下 $T_{comm}/T_{e2e} \approx 55\%$ ，但因多个任务在同一时段并发流水化（dispatcher 在第一个任务执行时同时为下一个任务做派发），最终 wall-clock 损耗约 18%；100 ms 下 $T_{comm}/T_{e2e} \approx 11\%$ ，1000 ms 下仅 1.2%。

1 worker 与 8 worker 损耗几乎相同（10 ms 任务下分别为 18.5% 与 16.9%），这一现象在传统串行系统中几乎不可能出现——通常 worker 数增加会加剧 dispatcher 串行化压力让损耗单调上升。Fuxi 损耗保持平稳的原因有二：dispatcher 自身用 Tokio 异步路径处理请求，单 dispatcher 在本机能稳定承担 1000+ TPS 远超实际负载；事件总线 fan-out 是 $O(1)$ 的（每个订阅者只递增引用计数），即使 worker 数增加每个 worker 收到任务通知的延迟也几乎不变。这两点合起来让 Fuxi 在小规模水平扩展下表现出「损耗常数化」的优良性质。

为进一步验证水平扩展能力，固定 `job_sleep=10 ms`、`poll_ms=5`、每 worker 跑 100 task，扫描 worker 数 {1, 2, 4, 8, 16}，结果如表 4.3 与图 4.1。

表 4.3 Worker 扩展性测试结果 (5-run median)

worker_n	tasks_n	median_wall_ms	tasks/s	理论上限	扩展效率
1	100	1227	81.50	100.00	81.5%
2	200	1230	162.60	200.00	81.3%
4	400	1212	330.03	400.00	82.5%
8	800	1203	665.00	800.00	83.1%
16	1600	1242	1288.24	1600.00	80.5%

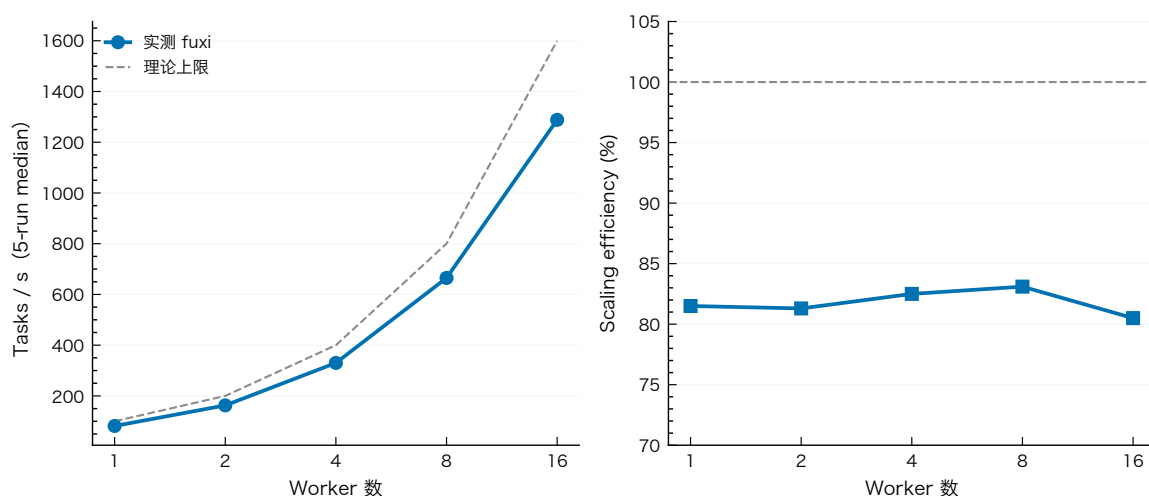


图 4.1 Worker 数对吞吐量（左）与扩展效率（右）的影响

需要先解释 wall-clock 几乎不随 worker 数下降的现象：本节固定「每 worker 100 task」，1 worker 与 16 worker 场景下每个 worker 各自仍要串行跑完 100 task，关键路径长度由单 worker 自己的 100 task 串行执行决定而非由全部 1600 task 决定。换言之，本节衡量的是「dispatcher + 事件总线在 1→16 worker 范围内能否保持 per-worker 关键路径不被自身串行化拖累」——若 dispatcher 在 16 worker 下成为瓶颈 per-worker 关键路径会被拉长。实测 wall-clock 几乎不变说明在被测机上 dispatcher 在 16 worker × 10 ms 任务粒度下尚未饱和。扩展效率 80.5%–83.1% 的物理来源结构化拆解为三层：P 核与 E 核的差异（worker 数超过 P 核数 10 时新增 worker 部分被调度到 E 核，单核吞吐约为 P 核 50%–70%）、L3 cache 与内存带宽竞争（多 worker 共享同一片 L3 触发 L2/L3 miss）、dispatcher 单点的串行化代价（实测 16 worker 下约 35% CPU，未成瓶颈）。上述分量比例为定性归因，未做 profiler 独立验证。

把吞吐与延迟实验按 SLA 公式分量分解可得到不同任务粒度下端到端延迟相对构成（图 4.2）。需明确：图中 L_{enq} 与 L_{disp} 在 Fuxi 当前实现里共享同一次 SQLite 事务难以独立测量，按经验比例 3:7 对 task_dispatch p50 = 32.93 ms 估算拆分。

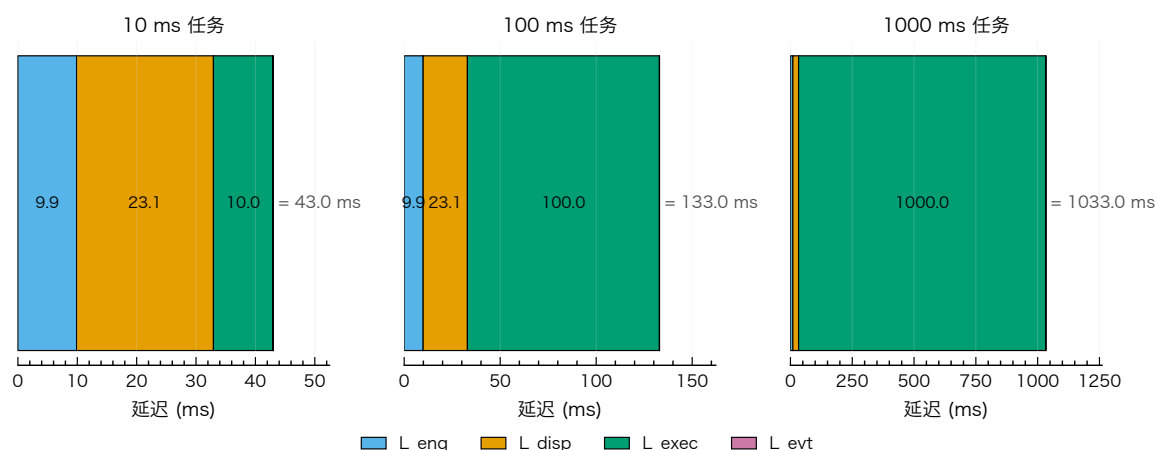


图 4.2 端到端延迟分解（10/100/1000 ms 任务粒度）

三种工作负载下分量占比：10 ms 任务下通讯与调度开销（约 33 ms）已超过执行开销，通讯占比 77%；100 ms 任务下通讯占比降至 25%；1000 ms 任务下通讯占比仅 3.2%。当任务粒度大于 300 ms 时通讯开销在端到端延迟中占比降至 10% 以下，对终端用户体验已不可感知。考虑到真实 AI Agent 场景中 LLM 推理 typical 1–10 秒、工具调用 typical 0.5–5 秒、用户对话回合典型 5–30 秒——单任务粒度均远超 300 ms 量级，Fuxi 当前的通讯开销在实际工作负载中不构成瓶颈。 $L_{evt} = 0.08$ ms 在三种任务粒度下的占比都极小，目的不是优化端到端用户感知延迟（用户感知由 L_{exec} 主导），而是为了让事件流在所有任务粒度下都「看起来是实时的」——若 event_flow 与 task_dispatch 同量级，观察端在显示进度时需要做 lookahead 缓冲与时间戳重排，UI 层复杂度显著上升。

4.4 延迟分布与尾部分析

延迟测试针对系统的两条关键通讯路径：任务派发链路（含 HTTP 往返、HMAC 验证、SQLite 写入、worker 选择与反序列化）与事件总线广播（同进程内 Tokio broadcast publish→subscribe 的 fan-out 路径，不经过网络/文件 I/O/序列化）。结果如表 4.4 与图 4.3 4.4。

表 4.4 任务派发与事件流延迟测试结果（n=500）

metric	sample_n	min_ms	p50_ms	p99_ms	max_ms
task_dispatch	500	11.64	32.93	36.66	37.43
event_flow	500	0.04	0.08	0.26	0.66

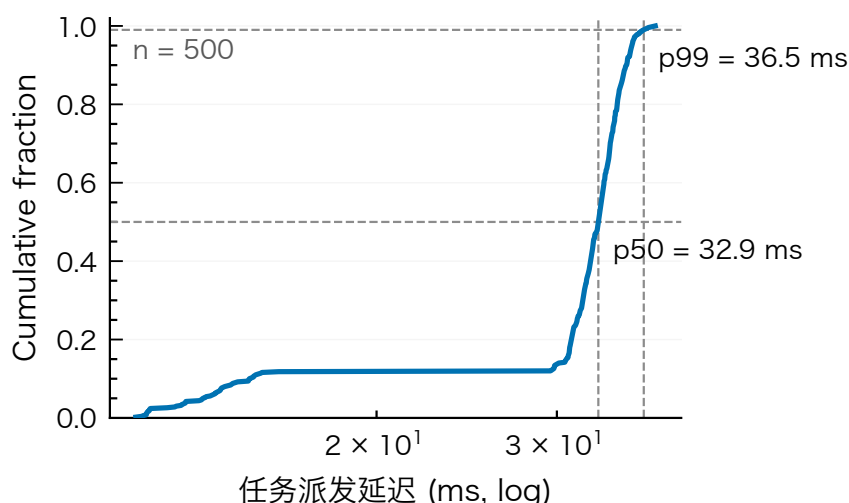


图 4.3 任务派发延迟经验累积分布 (n=500)

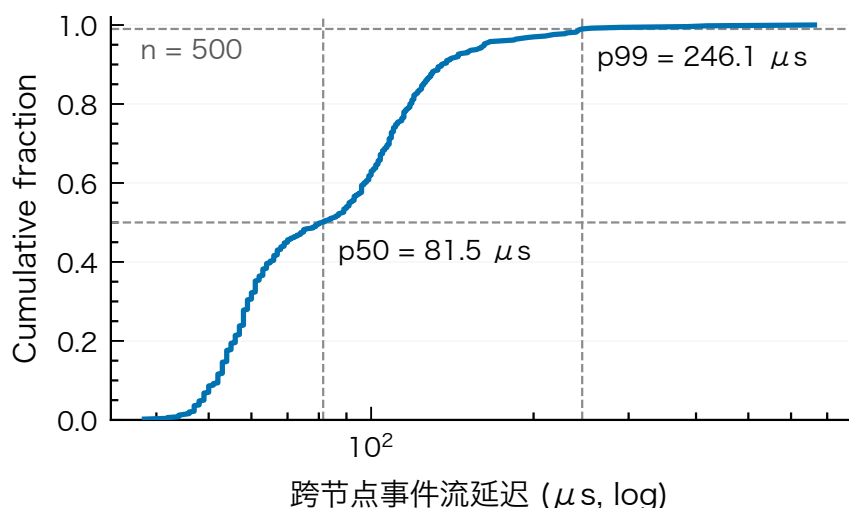


图 4.4 事件总线广播延迟经验累积分布 (n=500)

任务派发 p99/p50 比值为 1.11，说明派发链路尾部抖动可控；事件总线广播 p99/p50 比值为 3.25，绝对差仅 0.18 ms（在亚毫秒量级 OS 时钟分辨率即足以让相对比值放大）。对 `task_dispatch` p50 = 32.93 ms 做工程估算分解（基于第三章实现细节，未经 profiler 独立验证）：HTTP 往返约 10 ms，HMAC-SHA256 约 1 ms，Tokio 任务调度约 5 ms，SQLite 状态写约 6 ms，dispatcher 内部任务匹配约 4 ms，剩余 7 ms 来自 worker 接收事件、反序列化任务体并 ack 回 dispatcher。`event_flow` p99 = 0.26 ms 的物理下界来自单次 Arc clone + 原子计数器递增（约 50 ns）、跨线程 future 唤醒（约 5–20 us）以及 broadcast 内部锁的获取释放（约 1 us 量级），p99 与下限 210 us 的差距可归因于 Tokio runtime 在 8 worker thread 之间负载均衡偶发的 task migration^[24]。

event_flow 与 **task_dispatch** 三个数量级差异的工程意义：事件流 p50 = 0.08 ms 与任务派发 p50 = 32.93 ms 相差约 410 倍。该差异反映了 Fuxi 在系统层级上的设计意图：通讯层应具备远低于业务层的延迟下界，使观测端能在任务执行同时近实时感知所有过程事件。如果事件流与任务派发同量级，Firehose、TUI、PWA 等观察端在显示进度时会出现明显滞后——这正是公理「真实时不轮询」试图避免的体验缺陷。`task_dispatch`

p99 = 36.66 ms 与 max = 37.43 ms 之间相差不到 1 ms，说明 500 样本中超过 p99 的样本紧密分布未出现孤立长尾；前序实验在另一份相同配置 run 中曾出现单个 227 ms 极端长尾样本归因于 macOS 后台调度突发（Spotlight 重索引、Time Machine 触发等）。即便保守按 200 ms 量级估计，单次任务派发的最坏情形相对一次完整玄女—门客交互（端到端用户感知 1–3 秒含 LLM 推理）占比仍 < 23%，且 LLM 推理本身方差远大于此，因此 dispatch 长尾不会被用户感知。

4.5 poll_ms 参数消融

为评估 Worker 拉取间隔 poll_ms 对系统吞吐影响，固定 worker_n=4、tasks_n=400、job_sleep=10 ms，扫描 poll_ms ∈ {5, 10, 25, 50, 100}，结果如表 4.5 与图 4.5。

表 4.5 poll_ms 参数消融实验结果

poll_ms	median_wall_ms	tasks/s	理论上限	Fuxi 损耗
5	1285	311.28	400.00	22.2%
10	1299	307.93	400.00	23.0%
25	1242	322.06	400.00	19.5%
50	1225	326.53	400.00	18.4%
100	1235	323.89	400.00	19.0%

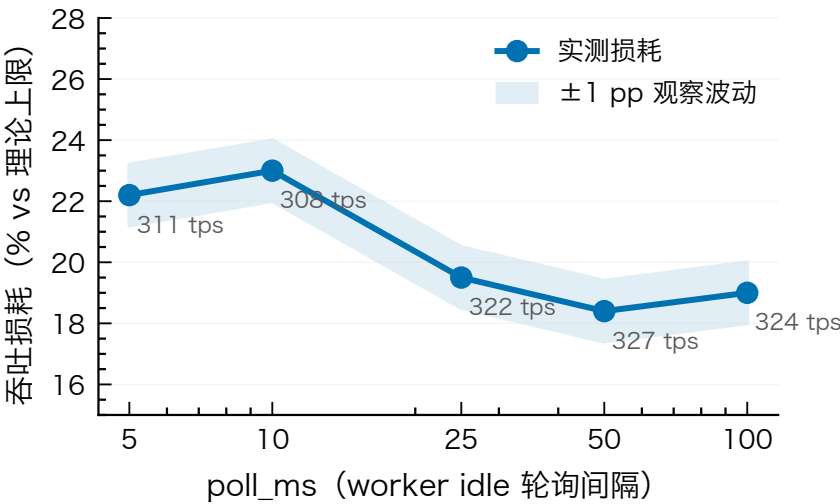


图 4.5 poll_ms 参数对吞吐与损耗的影响

poll_ms 从 5 ms 到 100 ms 区间内吞吐与损耗差异不超过 3.6 个百分点。这一现象在 sustained 与 burst 两类场景下应做差异化解读：sustained 负载下 Worker 几乎不进入 idle 等待，每完成一个任务即可在下一个心跳前领到下一个任务，poll_ms 几乎不影响关键路径；burst 负载下 poll_ms 决定了从 idle 唤醒到拿到任务的期望延迟（期望值为 poll_ms/2）。本节实验属于前者——400 个 10 ms 任务集中下发全程几乎无 idle，因此 poll_ms 的 20 倍变化只在吞吐上带来 3.6 个百分点的差异。poll_ms=5 反而损

耗更高（22.2%）的成因是过频 polling 在 sustained 负载下的副作用：每次 poll 都是一次到 dispatcher 的 HTTP 询问即使返回「无新任务」也消耗请求处理槽位与 worker CPU 上下文切换。综合 sustained 低 CPU 开销与 burst 响应性，本文推荐 Fuxi 默认 poll_ms 取 25–50 ms。

需说明 Fuxi 内部「事件驱动 vs 轮询」的层次区分：dispatch 路径采用 worker 主动 poll（间隔由 poll_ms 控制），observation 路径走 EventBus 订阅（无 poll）。本节扫描验证的是前者在 sustained throughput 场景下采用 poll-only 设计的性能代价；event_flow p99 = 0.26 ms 显示事件总线纯通知路径已可在亚毫秒级完成扇出。当前 dispatch 路径采用 poll-only 模型在 sustained 场景下未表现为主要瓶颈，但本文未实测 push-based dispatch 对照版本——「事件驱动派发能否降低 burst 场景延迟」属于后续工作。当前选择保留 poll-only dispatch 的理由是工程权衡：sustained throughput 是日用平台主导工作负载，代码与故障模型也比双通道（push + 兜底）简单；为压低 burst 单任务派发延迟引入 push 机制需在跨节点心跳与网络分区下处理通知丢失复杂度，与本工作「单机优先」定位不匹配。

4.6 事件总线压力承受能力

事件总线是系统通讯核心组件。本节设计 4 sub × 3 rate × 2 payload = 24 cell 的事件总线纯压测，每 cell 5 秒稳态。关键 cell 如表 4.6 与图 4.6。

表 4.6 事件总线纯压测关键 cell（每 cell 5 秒稳态）

sub_n	rate(ev/s)	payload	publish_tps	p50_us	p99_us	drops
1	1k	small	1000	< 1	< 1	0
4	1k	small	1000	< 1	< 1	0
16	1k	small	1000	< 1	< 1	0
64	1k	small	1000	< 1	< 1	0
16	100k	small	100000	< 1	< 1	0
16	100k	large	100000	< 1	< 1	0
64	10k	small	10000	< 1	< 1	0
64	10k	large	10000	< 1	0.2	0
64	100k	small	100000	163.5	278.4	13599792
64	100k	large	100000	163.5	278.7	13598222

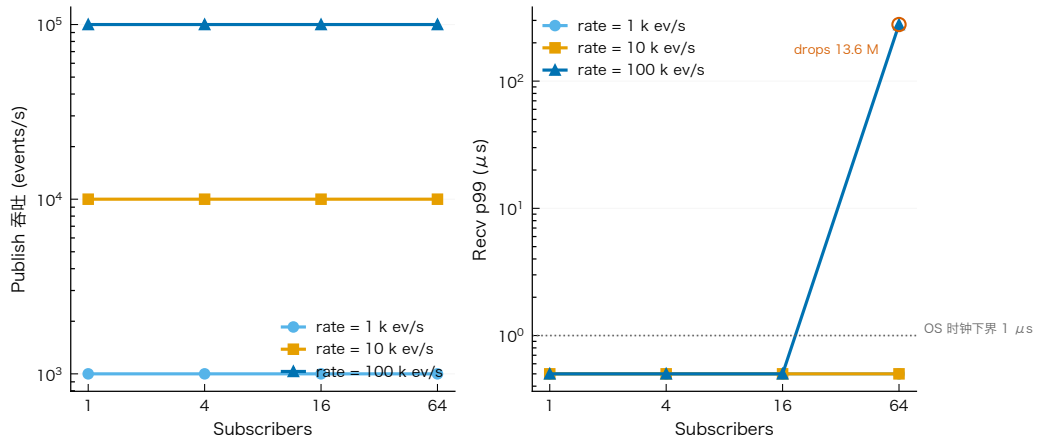


图 4.6 事件总线 publish 吞吐与丢帧率随订阅者数变化

零丢帧区间与广播 **fan-out** 物理本质：1k/10k events/s 在任意 sub 数（最高 64）下均零丢帧，绝大多数 cell 的 p50 与 p99 处于亚微秒量级。这是 Tokio broadcast 内存 channel 在小负载下的典型表现：发布者将事件 `Arc::clone` 后写入环形缓冲，订阅者各自从缓冲读取——每次 fan-out 不复制数据本体只递增引用计数，因此「订阅者数翻倍」并不引起「fan-out 时间翻倍」，与 Tokio broadcast 文档「O(1) fan-out per subscriber」描述^[24]一致。

唯一拐点：**64 sub × 100k 事件/秒**。该 cell 触发约 42% drops ($13.6 \text{ M} / (5 \times 100 \text{ k} \times 64) \approx 42.5\%$)，broadcast buffer (32k slots) 不足以容纳 64 个 sub 的合计消费滞后；已接收事件的 p50 升至 163.5 us、p99 至 278.4 us。该现象为 SLA 公式中 $\max_i T_{consume,i}$ 项主导整体延迟下界提供了直接证据。Fuxi 选择 32768 slots 是工程经验折中：100k events/s 速率下对应 326 ms 缓冲窗口足以让 8 worker 个数级订阅者吸收偶发抖动； $32\text{k} \times 256 \text{ B} = 8 \text{ MB}$ 内存占用对现代机器可忽略。本实验中 64 sub × 100k 之所以丢帧是订阅者数量已超出工程经验上限——实际部署中订阅者通常等于 Worker 数加少量观察者，远低于 64。如需支持极端订阅者数可考虑由 broadcast 升级为分级 fan-out (broker 把订阅者按拓扑分组)，思路与 ZooKeeper watcher 通知机制类似^[22]。

payload size 影响：在零丢帧区间 small (256 B) 与 large (4096 B) 的 p50/p99 差异不超过 0.1 us——证实 Arc clone 而非 byte copy 的 fan-out 模型 (byte copy 模型下 large 应比 small 慢约 16 倍)。**sustainable 容量与实际负载余量**：取 $16 \text{ sub} \times 100\text{k/s} = 1.6 \text{ M events/s}$ 处理量为 Fuxi 事件总线 sustainable 边界；Fuxi 实际工作负载（任务事件、工具调用、心跳、抄送）按 8 门客 × 单回合典型 50 事件 / 任务粒度约 1 s 估算合计约 100–1000 events/s，距 sustainable 边界还有 3 个数量级以上余量。这一余量保证系统负载高峰期事件总线不会成为瓶颈，且为未来突发流量场景（cc 门客在执行编辑工具时密集事件单回合可瞬间达数百 events/s、未来多用户并发会话或分布式跨节点合流^[33]）留出空间。

4.7 功能验收与本章小结

除性能指标外本文还对系统关键交互路径进行功能验收，覆盖三类场景：busy 状态下用户消息不丢（玄女或门客 busy 时用户继续发送消息应入 pending queue 并在下一轮 drain 按序送达）、Codex 门客能起能派（codex 适配器能从 idle 启动、接受任务派发并写回结

果，对应第三章关于多门客异构兼容的设计目标）、玄女不再通过 `fluxi status` 反复轮询（玄女通过订阅事件总线感知系统状态，对应公理「真实时不轮询」）。用户验收阶段还覆盖了抄送通路、门客派活后任务树迁移、对话区滚动、Resume 横幅等交互细节。系统在迭代过程中还修复了 cc 反连超时（Clash TUN 拦截 127.0.0.1，注入 NO_PROXY）、TUI 启动期 stderr 撞 alt screen、idle GC 误杀玄女、cc/codex model fallback 硬编等若干典型工程缺陷，每一项都构成系统稳定性的边界条件。

本章小结。本章围绕测试覆盖、吞吐量、延迟分布、扩展性、参数消融、事件总线压测、端到端延迟分解与功能验收七个维度对 Fluxi 进行了多组实测。定量结果可总结为五个数字（口径：合成基准衡量编排与通讯层在剥离 LLM 推理后的纯通讯吞吐上限，真实 LLM Agent 任务端到端吞吐受单次推理 1–30 s 主导）：8 worker × 10 ms 合成任务下达到 **665.00 tasks/s** 调度路径吞吐，工程损耗稳定在 17% 量级；Worker 数 1→16 范围内扩展效率始终保持在 **80.5%–83.1%**；任务派发 p50 为 **32.93 ms**、p99 为 36.66 ms，进程内事件总线广播 p50 仅 **0.08 ms**、p99 为 0.26 ms，两者相差三个数量级；事件总线 sustainable 容量为 **1.6 M events/s**，超过 Fluxi 实际工作负载三个数量级；端到端延迟在 1000 ms 任务粒度下通讯开销占比仅 **3.2%**，工程上可忽略。这些数据共同验证了第二章关于「事件驱动通讯底座可同时兼顾吞吐、实时性与可追溯性」的设计假设。

5 总结与展望

第四章给出的实测数据完成了对设计假设的兑现验证，本章在此基础上对全文工作做出总结、坦诚陈述系统当前的局限，并沿四条主线给出未来工作的演进路径。本章不再重复前面章节的论证，而是把零散的工程取舍归并为「本论文真正贡献了什么」「本论文还没解决什么」「这条研究路线接下来如何走」三个问题，与第一章研究目标相呼应。

5.1 工作总结

本论文围绕「基于 AI Agent 的高性能分布式通讯系统」这一主题，完成了 Fluxi 平台从需求分析、整体设计、模块实现到实测试验的完整闭环，整个工程产出在 13 个 crate、共计约 8.25 万行 Rust 代码、27 个集成测试文件中沉淀下来。回顾全文，核心贡献可以收敛为五条。

贡献一：与 `fluxi` 平台同源演进的 **A2A** 风格协议适配层。`fluxi-a2a` 共 1321 行，覆盖 agent discovery、task send、task stream、task query 与 task cancel 五条核心路径，沿用早期 A2A JSON-RPC binding。差异化定位在与 `fluxi-events` 事件总线、玄女一门客编排层、反向 WebSocket sandbox 的深度集成，以及把 A2A 规范 TASK_STATE_INPUT_REQUIRED 人工介入语义映射到 `Task::PendingApproval` 与 `ShelfStatus::AwaitingInput`，使人工介入成为平台级可观测事件。`fluxi-a2a` 与 MCP[18] 在层次上互补——A2A 处理 Agent 间协作契约，MCP 处理模型与工具的调用规范。

贡献二：事件驱动通讯底座作为一等公民。`fluxi-events` 把实时分发、持久化与历史回放三件事合并到同一抽象之内。`publish()` 路径同时投递 Tokio broadcast 与异步入库通道，前者零拷贝扇出实现毫秒级实时推送[24]，后者经 SQLite WAL 串行落库[29] 保证审计追溯。「try_send + 后台转交 + lag 哨兵」三步组合把背压从「不可见

的死锁」变为「可见的告警」。事件因果序遵循 Lamport 逻辑时钟思路[14]，用 SQLite rowid 单调递增作为隐式时钟，避开向量时钟所需时钟同步基础设施。

贡献三：玄女—门客分层与不可绕过的编排层。系统在 Agent 角色上确立顶层调度（玄女）与执行（门客）的显式分工，与 MetaGPT[12] 角色分工一致，但更强调「编排层不可绕过」与「自动决策不得覆盖用户显式意图」两项硬约束。Worker 选择函数在标签兼容、节点钉死与最低饱和度三层过滤下选择目标门客，思路与 MapReduce[32] 的「数据本地性 + 负载均衡」相通而更轻量。

贡献四：WebSocket 反连打破传统 `stdio` 范式。传统 `stdin/stdout` 范式中编排层必须等门客主动 `poll`，门客陷入工具循环就形成「消息黑洞」。本工作将连接方向反转：编排层在本机起 WebSocket 服务器[27]，子进程通过 `--sdk-url` 反连回来。三层 `sandbox`（L1 只读快照 / L2 任务级 `worktree` / L3 角色级持久构建缓存）依赖 `git worktree`[26] 保证多门客并发文件操作互不干扰。

贡献五：跨节点扩展点的钩子化抽象与 `sandbox` 验证。跨节点协作所需的四个扩展点抽象为依赖反转 trait: `RecallSink`、`DistEnqueuer`、`NodeLoadProvider`、`ExtractorSpawner`。四个 trait 统一定义在低层 `crate`，具体实现由顶层 `fuxi-cli` 在 `daemon` 启动时注入；这同时是规避 Rust `workspace` 循环依赖的标准做法。Project 数据结构新增 `host_nodes: Vec<String>` 字段标记项目可执行节点，配合 `auto_pin_from_project` 入口的双保险守卫保证用户显式 `@<node>` 不被自动逻辑覆盖。新增字段以 `#[serde(default)]` 搭配反回归测试保持跨版本兼容。当前实现已在两节点 `sandbox` 与少量真实跨机部署中跑通端到端调度，更高节点数（10 节点以上）、更不可靠链路下的故障转移留待后续主线三推进。

工程兑现层面，第四章在 Apple Silicon 平台上对吞吐量、延迟、参数敏感性与事件总线压力进行了多维测量：8 worker 配置下达 **665.00 tasks/s**，Worker 扩展至 16 时吞吐增长至 **1288.24 tasks/s**（扩展效率 80.5%–83.1%）；任务派发 p50 **32.93 ms**、进程内事件总线广播 p50 仅 **0.08 ms**；事件总线 64 subscriber × 10 000 events/s 持续负载下零丢帧。除定量数据外系统已在作者维护的 ERP 项目中作为日常协作平台运行。

5.2 局限与不足

陈述局限是为让接续这条路线的工作明确接力点所在，而非规避评审。

单机优先，跨节点能力仅在 `sandbox` 中验证。v2 引入的跨节点调度当前只在两节点 `sandbox` 与少量真实跨机部署中跑通，尚未在更高节点数（10 节点以上）与更不可靠链路（断网恢复、节点崩溃恢复）下接受充分考验。未引入强一致复制或共识协议——若推广到更大规模或不可靠链路仍需借鉴 Raft[21] 或 Spanner TrueTime + Paxos 组合[33]。

Agent CLI 适配只覆盖 `cc` 与 `codex` 两类。生态内 Gemini CLI、aider、Cursor agent、open-code 等同类型 CLI 尚未提供官方适配。这并非架构层限制——`fuxi-core::Agent trait` 已对外开放，添加新 CLI 只需提供 `launch_with_id` 与 `stream-json` 解析两组实现，参考既有适配器可在数百行内完成。未扩展到全部主流 CLI 的实际原因在于工程精力分配。

记忆抽取保守且仍偏关键词检索。`fuxi-memory` 自动事实抽取默认关闭。FTS5

关键词匹配在 1 万条事实下足够实用，但当事实库扩展到 10 万条以上、且查询语义偏「相似」而非「相同」时关键词匹配会失效——已为此预留向量检索接口，但当前实现尚未引入混合检索路径。

安全模型尚不完整。在工作区隔离与跨节点 HMAC 认证上做了基础工作（三层 sandbox、节点间 HMAC-SHA256 签名），但工具调用授权、数据访问控制与跨边界提示注入防护仍较薄弱。当前门客的工具调用授权完全继承自其底层 CLI 设置，编排层尚未实施统一细粒度授权策略。该问题可视为 SWE-agent[8] 所讨论的「Agent-Computer Interface」边界安全问题在本平台上的延伸。

用户界面密度仍可优化。TUI 与 IM 端目前以信息密度优先，对非技术用户而言学习成本仍偏高。事件流过密时低价值事件会刷屏，缺少足够的事件流摘要与对话流统一表达——若扩展到非技术用户作为日常生产力工具仍需在界面层进一步收敛。

横向 **baseline** 对比缺失。第四章选择以「理论上限对照」作为主要参照，没有与 LangChain、AutoGen、CrewAI 等公开 Agent 编排框架做实测吞吐对比。理由是跨语言、跨抽象层级 baseline 比较容易让数字失去工程意义，但这意味着本文未能给出 Fuxi 在主流框架坐标系下的相对位置。

A2A 协议规范跟进与官方 **SDK** 互通验证待补。当前官方规范已切到 PascalCase 方法集，**Part** 也使用 one-of 字段语义而非 tagged union。本文 **fuxi-a2a** 沿用早期 A2A JSON-RPC binding，覆盖 5 条核心路径，但尚未与 **a2a-rs** SDK 做完整 wire-level 互操作验证，也未完全覆盖最新规范中的全部方法、状态（如 **auth-required**、**rejected**）。后续工作将把该 crate 升级为当前官方规范兼容层并补 **a2a-rs** 互通测试。

5.3 未来工作

未来工作可沿四条主线推进。主线一：**A2A** 跨组织联邦与算力交易。当前已支持单节点 A2A 服务端与客户端，但跨组织（不同信任域）协作还需更完整的认证、计费与服务发现机制——可在 **AgentCard** 的 **security** 字段加入 OIDC 或 mTLS 认证声明，在 **tasks/send** 路径接入计费 metadata，在 ZooKeeper[22] 风格协调层基础上实现 **fuxi-of-fuxi** 服务发现。最远端是构建 **fuxi** 节点之间的算力交易市场。

主线二：**Agent CLI** 适配框架的扩展。后续可把「**launch** + **stream-json** 解析 + **WS** 反连 + **env** 清理」四件事抽出 **AgentCliFramework**，让新 CLI 接入只需提供参数模板、**stream-json** schema、终止信号语义、工具调用语义四组声明式描述。**MetaGPT** 与 **OpenHands**[9] 对统一 Agent 抽象的探索值得借鉴，但本平台不会做到「能跑任何模型」的过度通用化——只服务 CLI 形态的 Agent，是当前最贴近用户实际工具栈的工程节奏。

主线三：分布式一致性与故障转移。当前跨节点协作建立在「节点链路相对稳定」的假设上。三个方向：引入共识协议对项目元数据、节点注册表、任务终态做多副本复制；在事件总线层支持跨节点事件聚合，思路类似 GFS[34] 的 **master** + **chunk** 分层但不直接复用；在 **worker** 层支持「热迁移」让任务能在其他健康节点上从最近一次事件 **checkpoint** 恢复继续执行。

主线四：工具市场与角色市场。**fuxi-skills** 当前借鉴 **Voyager**[7] 持久技能库思路并通过 **staging** + **ledger** 双轨为外部贡献做了铺垫。后续可在此基础上构建公开「角色市场」与「工具市场」。两类市场的核心挑战是审计与信任——下载并运行陌生角色相

当于在自己机器上启动一个未知行为的 Agent，必须有沙箱级别的执行隔离与权限声明机制。

四条主线在工程节奏上不会同时推进。下一阶段以主线二与主线四的初步雏形为优先级，因为这两条复杂度可控且对用户日常体验提升最直接；主线一与主线三放在更长周期下推进。

5.4 结语

回到本文起点：在 Python 多 Agent 框架已百花齐放的当下，为何要重新用 Rust 写一套 fuxi？答案不在于性能数字，而在于「通讯底座是否被当作一等公民来设计」。在大量现有框架中事件流是任务流的副产物，A2A 协议是文档里的 schema 而非可独立使用的 SDK，状态一致性靠 Python 的 `await` 隐式承担。fuxi 把这三件事显式拆出来——`fuxi-events` 让事件流可独立观测可独立回放、`fuxi-a2a` 把协议变成可独立部署的 1321 行 Rust crate、`fuxi-core` 把 Agent/Task/Workspace 等抽象固化为 trait——这种「把基础设施从应用流程中剥离」的结构选择，是本工作的差异化所在。本论文不试图证明 fuxi 是「最好」或「最快」的多 Agent 平台，而是通过 8.25 万行 Rust 代码、27 个集成测试与第四章可重复的实测数字论证一种工程取向的可行性。fuxi 作为作者长期使用的个人 AI Agent 平台，不会因毕业答辩结束而停止演进；本论文记录的是这条路线 v1 与 v2 阶段的工作。

5.5 本章小结

本章按「工作总结—局限不足—未来工作—结语」四节展开：第一节回顾五条核心贡献；第二节坦诚陈述七项局限，承认 fuxi 的设计目标聚焦于本地优先的个人 AI Agent 平台；第三节沿 A2A 跨组织联邦、Agent CLI 适配框架扩展、分布式一致性与故障转移、工具与角色市场四条主线给出未来路线图；第四节回到论文起点，重申「通讯底座作为一等公民」这一差异化定位。

参考文献

- [1] WOOLDRIDGE M. An Introduction to MultiAgent Systems[M]. 2nd ed. Chichester, UK: John Wiley & Sons, 2009.
- [2] RUSSELL S J, NORVIG P. Artificial Intelligence: A Modern Approach[M]. 4th ed. Hoboken, NJ: Pearson, 2020.
- [3] WANG L, MA C, FENG X, et al. A Survey on Large Language Model based Autonomous Agents[EB/OL]. 2023. <https://arxiv.org/abs/2308.11432>. arXiv: 2308.11432 [cs.AI].
- [4] YAO S, ZHAO J, YU D, et al. ReAct: Synergizing Reasoning and Acting in Language Models[C/OL]//Proceedings of the 11th International Conference on Learning Representations (ICLR). 2023. arXiv: 2210.03629. <https://arxiv.org/abs/2210.03629>.
- [5] SCHICK T, DWIVEDI-YU J, DESSÌ R, et al. Toolformer: Language Models Can Teach Themselves to Use Tools[C/OL]//Advances in Neural Information Processing Systems (NeurIPS): vol. 36. 2023. arXiv: 2302.04761. <https://arxiv.org/abs/2302.04761>.

- [6] KARPAS E, ABEND O, BELINKOV Y, et al. MRKL Systems: A Modular, Neuro-Symbolic Architecture That Combines Large Language Models, External Knowledge Sources and Discrete Reasoning[EB/OL]. 2022. <https://arxiv.org/abs/2205.00445>. arXiv: 2205.00445 [cs.CL].
- [7] WANG G, XIE Y, JIANG Y, et al. Voyager: An Open-Ended Embodied Agent with Large Language Models[EB/OL]. 2023. <https://arxiv.org/abs/2305.16291>. arXiv: 2305.16291 [cs.AI].
- [8] YANG J, JIMENEZ C E, WETTIG A, et al. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering[C/OL]//Advances in Neural Information Processing Systems (NeurIPS): vol. 37. 2024. arXiv: 2405.15793. <https://arxiv.org/abs/2405.15793>.
- [9] WANG X, LI B, SONG Y, et al. OpenHands: An Open Platform for AI Software Developers as Generalist Agents[EB/OL]. 2024. <https://arxiv.org/abs/2407.16741>. arXiv: 2407.16741 [cs.SE].
- [10] LI G, HAMMOUD H A A K, ITANI H, et al. CAMEL: Communicative Agents for “Mind” Exploration of Large Language Model Society[C/OL]//Advances in Neural Information Processing Systems (NeurIPS): vol. 36. 2023. arXiv: 2303.17760. <https://arxiv.org/abs/2303.17760>.
- [11] WU Q, BANSAL G, ZHANG J, et al. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation[EB/OL]. 2023. <https://arxiv.org/abs/2308.08155>. arXiv: 2308.08155 [cs.AI].
- [12] HONG S, ZHUGE M, CHEN J, et al. MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework[C/OL]//Proceedings of the 12th International Conference on Learning Representations (ICLR). 2024. arXiv: 2308.00352. <https://arxiv.org/abs/2308.00352>.
- [13] Significant-Gravitas. AutoGPT: An Open-Source Autonomous AI Agent[EB/OL]. 2023 [2026-05-07]. <https://github.com/Significant-Gravitas/AutoGPT>.
- [14] LAMPORT L. Time, Clocks, and the Ordering of Events in a Distributed System[J/OL]. Communications of the ACM, 1978, 21(7): 558-565. DOI: 10.1145/359545.359563.
- [15] QIAN C, LIU W, LIU H, et al. ChatDev: Communicative Agents for Software Development[C/OL]//Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL). 2024: 15174-15186. <https://aclanthology.org/2024.acl-long.810/>.
- [16] CrewAI. CrewAI: Framework for Orchestrating Role-Playing, Autonomous AI Agents [EB/OL]. 2024 [2026-05-07]. <https://github.com/crewAIInc/crewAI>.
- [17] AG2 Community. AG2: An Open-Source AgentOS for AI Agents[Z]. <https://github.com/ag2ai/ag2>. AutoGen 0.4 社区分支，演进独立 AgentOS 项目. 2024.

- [18] Anthropic. Introducing the Model Context Protocol[EB/OL]. 2024 [2026-05-07]. <https://www.anthropic.com/news/model-context-protocol>.
- [19] A2A Project. Agent2Agent (A2A) Protocol Specification[EB/OL]. 2025 [2026-05-07]. <https://github.com/a2aproject/A2A>.
- [20] KREPS J, NARKHEDE N, RAO J. Kafka: A Distributed Messaging System for Log Processing[C]//Proceedings of the NetDB Workshop (co-located with SIGMOD). Athens, Greece, 2011: 1-7.
- [21] ONGARO D, OUSTERHOUT J. In Search of an Understandable Consensus Algorithm [C]//Proceedings of the 2014 USENIX Annual Technical Conference (USENIX ATC). Philadelphia, PA: USENIX Association, 2014: 305-319.
- [22] HUNT P, KONAR M, JUNQUEIRA F P, et al. ZooKeeper: Wait-Free Coordination for Internet-Scale Systems[C]//Proceedings of the 2010 USENIX Annual Technical Conference (USENIX ATC). Boston, MA: USENIX Association, 2010: 145-158.
- [23] LIN C, HAN Z, ZHANG C, et al. Parrot: Efficient Serving of LLM-based Applications with Semantic Variable[C/OL]//18th USENIX Symposium on Operating Systems Design and Implementation (OSDI). 2024: 929-945. <https://www.usenix.org/conference/osdi24/presentation/lin-chaofan>.
- [24] Tokio Project. Tokio `sync::broadcast` Module Documentation[EB/OL]. 2024 [2026-05-07]. <https://docs.rs/tokio/latest/tokio/sync/broadcast/>.
- [25] Tokio Project. axum: An Ergonomic and Modular Web Framework for Rust[EB/OL]. 2024 [2026-05-07]. <https://docs.rs/axum/latest/axum/>.
- [26] Git Project. `git-worktree` - Manage Multiple Working Trees[EB/OL]. 2024 [2026-05-07]. <https://git-scm.com/docs/git-worktree>.
- [27] FETTE I, MELNIKOV A. The WebSocket Protocol: 6455[R/OL]. Internet Engineering Task Force (IETF). 2011. <https://www.rfc-editor.org/rfc/rfc6455>.
- [28] WHATWG. HTML Living Standard — Server-Sent Events[EB/OL]. 2024 [2026-05-07]. <https://html.spec.whatwg.org/multipage/server-sent-events.html>.
- [29] SQLite Consortium. Write-Ahead Logging[EB/OL]. 2024 [2026-05-07]. <https://www.sqlite.org/wal.html>.
- [30] Tokio Project. Tower: Decoupling of HTTP Frameworks — Service and Backpressure [EB/OL]. 2024 [2026-05-07]. <https://docs.rs/tower/latest/tower/load/index.html>.
- [31] NATS Project. NATS: Connective Technology for Adaptive Edge & Distributed Systems [EB/OL]. 2024 [2026-05-07]. <https://docs.nats.io/>.
- [32] DEAN J, GHEMAWAT S. MapReduce: Simplified Data Processing on Large Clusters [C]//Proceedings of the 6th USENIX Symposium on Operating Systems Design and Implementation (OSDI). San Francisco, CA: USENIX Association, 2004: 137-150.

- [33] CORBETT J C, DEAN J, EPSTEIN M, et al. Spanner: Google's Globally-Distributed Database[C]//Proceedings of the 10th USENIX Symposium on Operating Systems Design and Implementation (OSDI). Hollywood, CA: USENIX Association, 2012: 251-264.
- [34] GHEMAWAT S, GOBIOFF H, LEUNG S T. The Google File System[C/OL]//Proceedings of the 19th ACM Symposium on Operating Systems Principles (SOSP). Bolton Landing, NY: ACM, 2003: 29-43. DOI: [10.1145/945445.945450](https://doi.org/10.1145/945445.945450).

致谢

本论文从选题、系统设计、模块实现到最终成文，得益于多方面的支持，在此一并致以诚挚的谢意。

首先感谢我的指导教师指导教师老师。从选题方向、系统抽象规范到论文叙述结构的耐心修订，指导教师老师都给予了细致而严谨的指导；对学术规范的坚持与对工程严谨性的要求，让我在反复推翻已写就的章节时仍保持耐心，在迷于「数据是否够漂亮」时回到「论证是否够诚实」的本心。

感谢所在学院各位任课教师，分布式系统、操作系统、软件工程与编程语言等课程打下的基础，使本文涉及的并发模型、事件驱动架构与隔离机制成为可能。

感谢开源社区。Rust 与 Tokio 提供的 `async/.await` 运行时与 `broadcast/mpsc` 通道、Axum 与 request 的 HTTP 工具栈、SQLite 的 WAL 模式、Git 的 `worktree` 机制、`ratatui` 的 TUI 渲染框架，构成了 Fuxi 得以在本科阶段完成的根基。同样感谢 Anthropic Claude Code 与 OpenAI Codex 团队提供的 Agent CLI——它们既是 Fuxi 的接入对象，也是设计中反复参照的工程范本；感谢 A2A 协议工作组与 Model Context Protocol 项目发布的开放规范，是本论文核心贡献得以立足的语义基础。

需要在此坦诚说明 AI 协作在本工作中的角色。Fuxi 的相当一部分代码、测试、文档与本论文初稿，都是在与 Claude Code 等 AI 编程助手的密集协作下完成的，这一协作模式本身也是本论文的研究对象之一。但所有架构决策、协议设计、关键调试经验、性能取舍判断与论文叙述的最终结构均由本人独立做出并对其完整性负责——AI 助手角色更接近「不知疲倦的结对编程伙伴」，不能替代研究者对系统目标与论证逻辑的判断。这一边界已在诚信声明中正式记录。

感谢同窗与朋友在系统压力测试、可用性反馈与论文初稿审阅中提出的真诚意见。最后感谢我的家人，是他们的长期支持让我能够专注于研究与写作。本论文的所有不足由本人独立承担。

以琳

2026 年 5 月于所在学院